TANSZÉKVEZETŐ

DIPLOMATERVEZÉSI FELADAT

Vértesaljai Bálint (KL3N7K)
Mérnökinformatikus hallgató részére

Gaussian splatting integrációja Godot környezetben

Napjainkban igen komoly fejlődésen mentek keresztül az inverz grafikai módszerek, amelyek a képalkotás folyamatát megfordítva végeznek 3D rekonstrukciót képi információk alapján. Ezek között kiemelten nagy figyelmet kapott a 3D Gaussian Splatting (3DGS), amely a színteret egy speciális, gaussi normális eloszlásokkal kiegészített pontfelhőként reprezentált radiancia mező formájában rekonstruálja. Ezen reprezentáció renderelése igen hatékonyan elvégezhető, a klasszikus 3D modellek raszterezéséhez hasonló módon. A 3DGS reprezentáció igen széles körben kezd elterjedni, már ipari szabványok is kezdenek megjelenni [2].

Ezen korszerű technológia egy különösen érdekes alkalmazása grafikus assetek előállítása számítógépes játékokhoz, pl. a népszerű Godot engine-ben. Ehhez egyrészt szükséges a 3DGS reprezentáció integrációja a grafikai és fizikai motorba, másrészt a szokványos 3D modellekkel való interakció megvalósítása is.

Hivatkozások:

[1] Kerbl et al.: "3D Gaussian splatting for real-time radiance field rendering." ACM Trans. Graph. 42.4 (2023): 139-1.

[2] https://www.khronos.org/news/press/gltf-gaussian-splatting-press-release

A hallgató feladatai:

- Ismerkedjen meg az inverz grafika és a 3DGS technológia alapjaival.
- Végezzen irodalomkutatást a 3DGS-hez kapcsolódóan.
- Integrálja a 3DGS reprezentáció megjelenítését a Godot játékmotorba.
- Végezze el a 3DGS reprezentáció integrálását a Godot fizikai rendszerébe, biztosítsa a megfelelő interakciót a hagyományos 3D modellekkel.
- Implementálja szabványos, tömörített 3DGS formátumok importálását [2].
- Vizsgálja meg az instancing alkalmazási lehetőségeit 3DGS modellekre.
- Vizsgálja meg 3DGS modellek újra-világítható megjelenítésének lehetőségeit.

Tanszéki konzulens: Dr. Vaitkus Márton

Budapest, 2026. március 20.

Dr. Kiss Bálint
egyetemi docens
tanszékvezető

Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Irányítástechnika és Informatika Tanszék

H-1117 Budapest, Magyar Tudósok krt. 2. IE444.
Tel: +36 1 4632057, Fax: +36 1 4634112
www.iit.bme.hu • e-mail: bkiss@iit.bme.hu
e-mail: vaitkus@iit.bme.hu

Budapesti Műszaki és Gazdaságtudományi Egyetem

Villamosmérnöki és Informatikai Kar

Irányítástechnika és Informatika Tanszék

Gaussian splatting integrációja Godot környezetben

Diplomaterv

Készítette

Vértesaljai Bálint

Konzulens

Dr. Vaitkus Márton

2026. 12. 10.

Tartalomjegyzék

Kivonat **8**

Abstract **9**

1. Bevezetés **10**

1.1. Számítógépes grafika . . . 10
1.2. Inverz renderelés . . . 10
1.3. Gaussian Splatting . . . 10
1.4. Godot . . . 11
1.5. Gaussian Splatting a Godot-ban . . . 12
1.6. Motiváció . . . 12
1.7. A diplomaterv további szerkezete . . . 12

2. Irodalomkutatás **13**

2.1. Gaussian Splatting témájú cikkek . . . 13
2.1.1. 3D Gaussian Splatting for Real-Time Radiance Field Rendering . . . 13
2.1.2. Relightolhatóság . . . 13
2.1.3. Időbeliség . . . 14
2.1.4. Szabványos GLTF formátum . . . 14
2.1.5. Generatív AI és inverz módszerek a képszintézisben . . . 14
2.2. Gaussian Splatting integrációk más játékmotorokban . . . 14
2.2.1. Unity . . . 14
2.2.2. Unreal Engine . . . 14
2.3. Gaussian Splatting implementációk Godot-ban . . . 15
2.3.1. GodotGaussianSplatting 2Retr0-tól . . . 15
2.3.2. godot-gaussian-splatting haztro-tól . . . 15
2.3.3. godot-gaussian-splatting ReconWorldLab-tól . . . 16
2.3.4. godotGS klausi3D-től . . . 16
2.4. Választott módszer . . . 16

3. Technológiák bemutatása **18**

3.1. Fájlformátumok . . . 18

3.1.1. PLY . . . 18
3.1.1.1. Eredeti, nézetirány-függő színekkel . . . 18
3.1.1.2. Újravilágítható, PBR anyagtulajdonságokkal . . . 19
3.1.2. SPLAT . . . 20
3.1.3. GLTF/GLB . . . 20
3.1.4. SPZ . . . 21
3.1.5. SOG . . . 21
3.2. Konvertáló programok . . . 21
3.2.1. SuperSplat . . . 21
3.2.2. Mesh2Splat . . . 22
3.2.2.1. Linux port . . . 23
3.2.3. SplatTransform . . . 23
3.3. Használt programnyelvek ismertetése . . . 24
3.3.1. A GDScript nyelv . . . 24
3.3.2. A GDShader nyelv . . . 24
3.3.3. A GLSL nyelv . . . 25
3.4. Godot bővítésének lehetőségei . . . 25
3.4.1. Megjelenítés teljes képernyős quad-ként . . . 25
3.4.2. Megjelenítés compositor effect-tel . . . 26

4. Tervezés **27**
4.1. Követelmények . . . 27
4.1.1. Funkcionális . . . 27
4.1.2. Nem funkcionális . . . 27
4.2. Architektúra . . . 28
4.3. Projekt végrehajtása . . . 29

5. Önálló munka bemutatása **30**
5.1. Egyszerre több pontfelhő megjelenítése . . . 31
5.2. Pontfelhők transzformációja . . . 33
5.2.1. Kiterjedés irányainak transzformációja . . . 35
5.2.2. Gömbi harmonikusok transzformációjának megjavítása . . . 35
5.3. Automata objektum-megtalálás . . . 36
5.4. Mélységhelyes összekombinálás . . . 39
5.5. Addon készítése . . . 40
5.6. Pull vagy Push modell . . . 40

5.6.1. Dinamikus láthatóság . . . 42
5.6.2. Kifinomultabb dinamikus láthatóság . . . 42
5.7. Elsőosztályú polgárok . . . 42
5.7.1. Editor ikonok . . . 43
5.7.2. Betöltés, és tárolt adat szétcsatolása . . . 43
5.8. AMD bug kijavítása . . . 43
5.9. CompositorEffect-es megoldás vizsgálata . . . 43
5.10. Instancing . . . 43
5.10.1. Instancing hatásának mérése . . . 44
5.10.2. Pull Request . . . 44
5.11. Újravilágíthatóság . . . 44
5.11.1. Saját fényszámítással . . . 45
5.11.2. A Godot fényszámításával . . . 46
5.11.2.1. Javítás nem irányfény esetére . . . 48
5.12. Fájlformátumok implementálása . . . 49
5.12.1. SPLAT adatfájl adatainak beolvasása . . . 49
5.12.2. GLTF adatfájl adatainak beolvasása . . . 51
5.13. Kollaboráció a másik implementációval . . . 51
5.13.1. Ikonok, dinamikus láthatóság és instancing Pull Request . . . 52
5.13.2. GLTF import Pull Request . . . 52
5.13.3. Issue a gömbi harmonikusok helyes transzformációjáról . . . 52
5.14. Kitérő: Typst . . . 52
5.14.1. Rövid bemutatás . . . 52
5.14.2. Miért jobb . . . 52

6. Plugin alkalmazásai, eredmények **53**
6.1. Használati utasítás . . . 53
6.2. Autó szimulátor, mint példafelhasználás . . . 53
6.2.1. Stuttering bug . . . 55
6.3. When Hamsters Attack TD, mint példafelhasználás . . . 55
6.4. Asset library plugin . . . 56
6.5. Még mit lehetne értékelésnek, eredménynek? . . . 56

7. Összefoglaló **57**

Irodalomjegyzék **58**

Függelék **60**

A. További teendők a Dipterv 2-ig (végleges doksi leadásig) 60
A.1. Egyéb TODO-k 60

HALLGATÓI NYILATKOZAT

Alulírott *Vértesaljai Bálint*, szigorló hallgató kijelentem, hogy ezt a diplomatervet meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem. A Függelékben egyértelműen megjelöltem, hogy a mesterséges intelligencia eszközeit alkalmaztam-e a dolgozat elkészítéséhez; amennyiben igen, annak módját és mértékét a táblázatban közöltem. Tudomásul veszem, hogy a mesterséges intelligenciával generált tartalomért – annak mértékétől függetlenül – teljes felelősséggel tartozom.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK az interneten nyilvánosan hozzáférhetővé tegye, a munka teljes szövege pedig a BME Címtárban regisztrált személyek számára elérhető legyen. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Budapest, 2026. 12. 10.

Vértesaljai Bálint
hallgató

Kivonat

TODO

Napjainkban igen komoly fejlődésen mentek keresztül az inverz grafikai módszerek, amelyek a képalkotás folyamatát megfordítva végeznek 3D rekonstrukciót képi információk alapján. Ezek között kiemelten nagy figyelmet kapott a 3D Gaussian Splatting (3DGS), amely a színteret egy speciális, gaussi normális eloszlásokkal kiegészített pontfelhőként reprezentált radiancia mező formájában rekonstruálja. Ezen reprezentáció renderelése igen hatékonyan elvégezhető, a klasszikus 3D modellek raszterezéséhez hasonló módon. A 3DGS reprezentáció igen széles körben kezd elterjedni, már ipari szabványok is kezdenek megjelenni.

Ezen korszerű technológia egy különösen érdekes alkalmazása grafikus assetek előállítása számítógépes játékokhoz, pl. a népszerű Godot engine-ben. Ehhez egyrészt szükséges a 3DGS reprezentáció integrációja a grafikai és fizikai motorba, másrészt a szokványos 3D modellekkel való interakció megvalósítása is.

Abstract

TODO

Nowadays, inverse graphics methods have undergone a very serious development, which perform 3D reconstruction based on image information by reversing the process of image creation. Among them, 3D Gaussian Splatting (3DGS) has received a lot of attention, which reconstructs the scene in the form of a radiance field represented as a special point cloud supplemented with Gaussian normal distributions. The rendering of this representation can be performed very efficiently, in a way similar to the rasterization of classical 3D models. 3DGS representation is becoming very widespread, and industry standards are already starting to appear.

A particularly interesting application of this modern technology is the production of graphic assets for computer games, e.g. in the popular Godot engine. This requires, on the one hand, the integration of the 3DGS representation into the graphics and physics engine, and on the other hand, the implementation of interaction with conventional 3D models.

1. Fejezet

Bevezetés

1.1. Számítógépes grafika

A számítógépes grafika gyűjtőfogalom alá tartozik mindenféle olyan technológia, amely a számítógép képernyőjén 2D, illetve 3D modelleket képes megjeleníteni, alakzatokat, jeleneteket képes kirajzolni.[1] Ezek a játékipartól kezdve az orvosi képalkotásig sok helyen felhasználásra kerülnek. Erősen kapcsolódó területek még a gépi látás, amikor a számítógép a kész képet értelmezi, a 3D modellek manipulációja, illetve többek között a mesterséges intelligencia modellekre, képekre történő alkalmazása is.

1.2. Inverz renderelés

Inverz renderelésnek azt a módszert nevezzük, amikor már kész képekből állítunk elő 3D modelleket. Tulajdonképpen megpróbáljuk rekonstruálni, milyen alakzat volt ott. [1] Ezek a kész képek lehetnek például egy adott objektumról készült fotók sok, különböző szögből, vagy egy beltér körbevideózva. Ez a rekonstrukció egy bonyolult folyamat, és sokféle módszer létezik rá. A különböző inverz renderelési módszerek napjaink egyik erősen kutatott témáját teszik ki. [2]

1.3. Gaussian Splatting

A Gaussian Splatting egy inverz rendering módszer, aminél a jelenetet Gauss-i eloszlású átlátszósággal rendelkező 3D foltokkal próbáljuk a lehető legjobban leírni [2]. Képekből előállítani a 3D modellt elég erőforrásigényes, viszont a modellek megjelenítése megfelelő minőség esetén kifejezetten közeli tud lenni a valósághoz. Nagy előnye, hogy a megjelenítőt sima, standard grafikus csővezetékkel is lehetséges implementálni, és valós időben, 100-as nagyságrendű FPS-sel renderelhető.

[1] https://en.wikipedia.org/wiki/Computer_graphics_(computer_science)

Az 1. ábrán látható egy autó Gaussian Splatting alapon megjelenítve, a kép bal oldala jó minőségben, míg a kép jobb oldala olyan módon, hogy az egyedi, Gauss-i eloszlású 3D foltok külön-külön látsszanak. A kép a feladatkiírásból származik.

1. ábra: A Gaussian Splatting szemléltetése egy autó modellel

1.4. Godot

A Godot Engine[2] egy szabad és nyílt forráskódú játékmotor, és az elmúlt években jelentős növekedésnek örvendett.[3] Megtalálhatóak benne a legfontosabb eszközök egy játék fejlesztéséhez, viszont a teljes motor az ellenfeleihez képest egy nagyon könnyű, lightweight megvalósítással rendelkezik: a motor egyetlen futtatható fájl elindításával használható, installációt nem igényel, és a teljes mérete 200MB alatt van. Így indításkor, és használat közben is, alap feladatok esetén legalábbis sokkal gyorsabb.

A 4-es fő verzióban a következő generációs renderer Vulkan alapú, és ennek bővítésére, illetve a Vulkan-hoz történő viszonylag alacsony szintű hozzáférésre is különféle lehetőségeket nyújt, még a beépített scriptnyelvből is.

[2]https://godotengine.org/

[3]https://gameworldobserver.com/2026/07/29/unity-is-no-longer-the-leader-godot-has-become-the-most-popular-engine-at-gmtk-game-jam-2026

1.5. Gaussian Splatting a Godot-ban

A Godot rendererjei, architektúrája alapértelmezetten a szokványos, háromszöghálós modellek megjelenítésére van optimalizálva, a Gaussian Splatting nincs a hivatalosan támogatott feature-ök között. Harmadik féltől származó Gaussian Splatting megjelenítők, pluginok viszont folyamatosan jelennek meg az interneten, ezeket a 2.3 fejezetben részletezem.

1.6. Motiváció

A motorba történő megfelelő integráció esetén lehetőség lenne Gaussian Splatting modelleket rendes, háromszöghálós modellekkel együtt megjeleníteni, a háromszöghálós modellekhez hasonlóan a 3D térben transzformálni, és ezekkel akár demókat, prototípusokat, illetve eszközöket, játékokat készíteni.

1.7. A diplomaterv további szerkezete

A továbbiakban bemutatom a témában megtalálható forrásokat, implementációkat, és leírom az általam elvégzett munka tervét, majd a megvalósítását, és a megvalósítás felhasználását. Végül összegzem a leírtakat. TODO picit bővebben kifejteni

2. Fejezet

Irodalomkutatás

2.1. Gaussian Splatting témájú cikkek

2.1.1. 3D Gaussian Splatting for Real-Time Radiance Field Rendering

A Gaussian Splatting (GS) technológiát az [2] cikk vezette be, ami ezután felkapott téma lett. A szerzők többek között különböző Neural radiance field (NeRF)[4] implementációkhoz hasonlították a saját implementációjukat, túlszárnyalva azokat pontosságban, és valósidejű megjelenítésben. Az általuk készített megoldás nem csak megjelenítést tartalmaz, hanem a képekből történő paraméteroptimalizációt is, ami a GS modelleket, pontfelhők paramétereit állítja elő.

2.1.2. Relightolhatóság

Az eredeti cikk nézetiránytól függő színeket lehetővé tett a kiterjedt pontoknak, viszont ezek beégetett színek, azaz egy adott modell egy adott irányból mindig ugyanúgy fog kinézni, fényektől függetlenül. Ez egy teljes 3D jelenet rekonstrukciójánál egy teljesen jó eredményt adó megoldás, viszont egy-egy különálló, dinamikus transzformációval rendelkező modell esetén nem mindig ad az elvártnak megfelelő eredményt. Ez különösen fontos lehet egy játékmotor esetén, ahol általában fények is vannak egy jelenetben, és a GS modell nem odaillőnek tűnhet, ha nem reagál a fényekre az elvárt módon.

Ennek megoldására többféle lehetőség van, más-más előnyökkel és hátrányokkal. A [3] forrásban megjelölt program segít bizonyos fajta, újravilágítható pontfelhőket generálni háromszöghálós modellekből, ennek a fájlformátumnak a részletezése a 3.1.1.2 fejezetben található. Ezen kívül a [4] implementációban is található újravilágítás opció, ami jelentősen különbözik a fentebbi fájlformátum alkalmazásától.

[4] https://en.wikipedia.org/wiki/Neural_radiance_field

2.1.3. Időbeliség

Külön izgalmas feladat, ha időben is változik a felvett jelenet, ilyenkor a pontfelhő időbeli transzformációját is el lehet tárolni, és ugyanazt a mozgó jelenetet akár több szemszögből, vagy mozgó szemszögből visszajátszani. Különböző pontfelhők utólagos szerkesztésére is lehetőség van. Erről szól a [5] cikk.

2.1.4. Szabványos GLTF formátum

A Gaussian Splatting modelleket valamilyen módon el kell tárolni. Erre sokféle megoldás született: különféle kulcsszavakkal rendelkező `.ply` fájlok, `.splat` fájlok, és egyéb fájformátumok is használatban vannak. Ezek részletesebb bemutatása a 3.1. fejezetben szerepel. 2026 februárjában jött ki a témában egy GLTF-kiterjesztés szabvány [6] is, amit a Khronos Group kezel. Ennek előnye, hogy pontszerű pontok halmazaként a GS megjelenítést nem támogató programok is meg képesek jeleníteni a modelleket, a visszafelé kompatibilitás miatt. Ennek a szabványnak az implementálása a feladatkiírásomban is szerepel.

2.1.5. Generatív AI és inverz módszerek a képszintézisben

A témában jól összegyűjtött információk forrása lehet többek között a Budapesti Műszaki és Gazdaságtudományi Egyetemen Dr. Vaitkus Márton által nemrég indított szabadon választható, Generatív AI és inverz módszerek a képszintézisben című tárgy honlapja és diasorai, amik a [1] forrásnál érhetőek el.

2.2. Gaussian Splatting integrációk más játékmotorokban

Gaussian Splatting implementációk a főbb játékmotorok mindegyikénél megjelent[5].

2.2.1. Unity

Nem található a Unity-hoz hivatalos GS támogatás, vagy hivatalos plugin. Található viszont több, harmadik fél által készített plugin: a UnityGaussianSplatting[6] kimondottan széles körben használt a csillagok mennyisége alapján, míg a gsplat-unity[7] frissebb, és szintén az előző kódbázison alapszik.

2.2.2. Unreal Engine

Unreal esetén sem találtam hivatalos, Epic által ösztönzött megvalósítást, található viszont ugyanúgy többféle harmadik fél által implementált megjelenítő.

[5] https://www.polyvia3d.com/guides/gaussian-splatting-unity-unreal
[6] https://github.com/aras-p/UnityGaussianSplatting/tree/main
[7] https://github.com/wuyize25/gsplat-unity

A Luma UE Plugin[8] és az XScene-UEPlugin[9] is egy szélesebb ökoszisztéma részeként jelenik meg, míg az unreal-splat[10] egy fapados, egyszerű implementáció benyomását kelti.

2.3. Gaussian Splatting implementációk Godot-ban

A Gaussian Splatting integráció minden népszerű játékmotornál egy releváns feladat. Én a témaválasztásnál a Godot Engine mellett döntöttem, mivel ez a motor áll hozzám a legközelebb, a szabad és nyílt forráskódú megvalósításával, és a könnyűségével, bloat-mentességével.

Mivel a motor, és a Gaussian Splatting téma is népszerű, ezért több, hasonló témájú implementáció is található, és a projektem lefolyása során folyamatosan jelennek meg az újabb megvalósítások. Külön feladat az ezen projektek felfedezése, és releváns esetben a kapcsolatfelvétel, kollaboráció.

2.3.1. GodotGaussianSplatting 2Retr0-tól

2Retr0 GS megjelenítője [7] az eredetileg leginkább használt implementáció. Ez egy darab GS modellt képes Godot-ban megjeleníteni, és egy repülő kamerával lehet körüljárni a modellt. Az 2. képen látható egy példakép, ami egy banánt ábrázol, az alatta levő terítővel együtt.

2. ábra: A GodotGaussianSplatting megjelenítő megjelenít egy banánt

2.3.2. godot-gaussian-splatting haztro-tól

Egy másik implementáció haztro-tól származik [8], amelyik viszont a projekt kezdetekor történő kipróbáláskor alulmaradt a 2.3.1. fejezetben taglalt

[8]https://lumaai.notion.site/Luma-Unreal-Engine-Plugin-0-41-8005919d93444c008982346185e933a1

[9]https://github.com/xverse-engine/XScene-UEPlugin/tree/main/UEPlugin

[10]https://github.com/JI20/unreal-splat

implementációtól. 2026-ban viszont további fejlesztéseket hajtottak rajta végre, ami a másik megjelenítőről nem mondható el.

2.3.3. godot-gaussian-splatting ReconWorldLab-tól

Ez az implementáció [4] már a diplomaterv írása közben bukkant fel. Ez már, az előzőekkel szemben, képes több modellt megjeleníteni, és transzformációt alkalmazni rájuk. Ez az plugin a 2.3.1. fejezetben bemutatott implementáción alapul. Az idő előrehaladtával több, előrehaladott fejlesztést is elvégzett a készítő: többek között többféle renderer támogatása, újravilágítás, és collision generálás.

2.3.4. godotGS klausi3D-től

Ez [9] egy szintén újonnan, 2026-ban feltűnt implementáció. Az architektúra viszont más, mint a fentebbieknél: ez a megvalósítás a teljes játékmotort forkolta, és kifejezetten nagy mennyiségű munka látható a repository-ban: több 100-as nagyságrendű issue-k, commit-ok és deployment-ek. Mindez annak ellenére, hogy az egyetlen contributor látható, és egyetlen csillaggal rendelkezik a repository. A sok befektetett energia a használatkor is érződik, például az extenzív mennyiségű debug információn, illetve az elérhető opciók, változtatható paraméterek számán is.

Ez a projekt kívülről meglehetősen AI-first projektnek tűnik, és az egyetlen ember által AI segítségével ennyi idő alatt elvégzett minőségi munka mennyisége elgondolkozásra, és kíváncsiságra késztet a szakma jövőjével kapcsolatban.

2.4. Választott módszer

Mivel a projekt kezdetekor a 2.3.3. és a 2.3.4. fejezetben leírt projektek még nem léteztek, ezért a választott módszer a 2.3.1-ben leírt projekt kibővítése úgy, hogy a feladatkiírásban szereplő integrációs problémák megoldásra kerüljenek.

Eközben, amint felbukkant a 2.3.3. fejezetben leírt projekt, kollaborációt kezdeményeztem [10], amelyet a másik fél elfogadott. Ezt megkönnyítette, hogy mindketten a 2.3.1-beli kódbázisból indultunk ki, így sok része a kódnak megegyezett. Némely fejlesztéseket én végeztem el, némelyeket ő, melyeket többé-kevésbé átvettünk egymástól. Ezt könnyen lehetővé tette az, hogy mindhárom kódbázis MIT licensszel rendelkezik, így lényegében a licensz és a copyright bemásolásával bármelyik kódot bárki felhasználhatja, még akár zárt forráskódú projektben is[11]. Így például a feladatkiírásban szereplő feladatok megoldásával is, több Pull

[11] https://en.wikipedia.org/wiki/MIT_License

Request-et készítettem neki. Ezeket az elvégzett munkákat, és kollaborációkat az 5. nagyfejezetben részletezem.

3. Fejezet

Technológiák bemutatása

A tervek és az önálló munka bemutatásához szükséges először a technológiai hátteret bemutatni. Ide tartoznak a pontfelhők lehetséges tárolási formátumai, a pontfelhőket manipulálni képes programok, illetve a Godot-ban szereplő programozási környezet.

3.1. Fájlformátumok

Ahhoz, hogy a pontfelhőket sorosítani tudjuk, elég pontonként az adott pont tulajdonságait eltárolni. Ezt az eltárolt paraméterek milyenségétől, a tömörítés jelenlététől és mértékétől, a pontok sorrendjétől, stream-elhetőségtől, illetve a formátum mögött álló cégektől, szervezetektől függően többféle szabvány szerint is megtehetjük. [11]

3.1.1. PLY

A `.ply` formátumot[12] először 1994-ben adták ki, és azóta is nagyrészt változatlan a keret. Könnyen lehet vele szöveges, vagy bináris formákban csúcsokat különböző, a fejlécben definiált tulajdonságokkal eltárolni.

3.1.1.1. Eredeti, nézetirány-függő színekkel

Egyszerűsége és flexibilitása miatt az eredeti [2] cikk a `.ply` formátumot választotta, így továbbra is ez az egyik legelterjedtebben használt formátum. A kiindulásként választott, 2.3.1. fejezetben taglalt program is ezt a formátumot támogatja.

A nézetirány-függő színekhez gömbi harmonikus (spherical harmonics, SH)[13] értékeket tárolunk el minden egyes ponthoz. A csúcsonként eltárolt értékek így, az összes szokásos nézetirány-függő paraméter használata esetén a következők [11]:

- 3 float: pozíció

[12]https://en.wikipedia.org/wiki/PLY_(file_format)
[13]https://en.wikipedia.org/wiki/Spherical_harmonics

- 3 float: felületi normál (nincs használva, de kötelező)
- 3 float: szín DC komponens (RGB)
- 45 float: szín gömbi harmonikus komponensek ($45 = 3 \cdot (3 + 5 + 7)$)
- 1 float: átlátszóság (szigmoid alkalmazása előtt)
- 3 float: skálafaktorok a 3 tengely mentén
- 4 float: elfordulás (kvaternió)

Ez a formátum így minden egyes pontonként (blob-onként) 62 lebegőpontos számot tárol el, így egy teljes jelenet könnyen meghaladja az 1GB méretet a lemezen. Ez veszteség (lossiness) és könnyű beolvashatóság szempontjából tökéletes, viszont elfoglalt tárhely szempontjából nem jó. Ezen kívül egy olyan formátumot használunk GS pontfelhők tárolására, ami nem erre lett tervezve.

3.1.1.2. Újravilágítható, PBR anyagtulajdonságokkal

Mivel a formátum könnyen bővíthető, módosítható, így egyszerűen lehetséges blob-onként más adatok eltárolása. Ezt használja ki a 3.2.2. fejezetben bemutatott program is, ami ugyanabban a formátumban, de más pontonkénti adatot tárol el, más céllal.

Ezekkel az adatokkal a színeket nem az adott blob-ra történő ránézés iránya határozza meg egyedül, hanem a jelenetben szereplő fények, és egyéb faktorok is. Ennek kiterjedt irodalma van a standard grafika pipeline esetén fizikai alapú renderelés (physically based rendering, PBR)[14] néven, mivel a háromszögeket is lehet hasonló anyagtulajdonságok figyelembe vételével megvilágítani. Ennek a technikának a segítségével tudjuk a pontfelhőket is újravilágítani.

A csúcsonként eltárolt értékek a következők ezesetben:

- 3 float: pozíció
- 3 float: felületi normál, ezesetben használva is van
- 3 float: szín DC komponens, albedó (RGB)
- 1 float: PBR metallic érték
- 1 float: PBR roughness érték
- 1 float: átlátszóság (szigmoid alkalmazása előtt)
- 3 float: skálafaktorok a 3 tengely mentén
- 4 float: elfordulás (kvaternió)

[14] https://en.wikipedia.org/wiki/Physically_based_rendering

Tehát a fő különbség a 3.1.1.1. fejezetben taglalt formátumhoz képest az, hogy az alapértelmezett adatok (transzform, RGB+A) mellett a +45 darab SH érték helyett +2 darab PBR, illetve +3 darab normál érték nyújtja a blob színének a jelenetbeli variációjának lehetőségét.

Itt fontos megemlíteni, hogy a 3.2.2-beli program könnyen előállítja a PBR értékeket a bemeneti mesh-ből, ezeknek az értékeknek a visszakövetkeztetése valós, inverz módszerrel felvett adathalmaz esetén viszont elképzelhetően sokkal körülményesebb, és pontatlanabb eredményt ad. Tudomásom szerint erre a problémára nincsen univerzális, könnyen használható, és széles körben pontos eredményt adó módszer. Ez nem a konkrét formátum negatívuma, hanem a teljes, 2.1.2. fejezetben taglalt újravilágításnak a megoldandó problémája.

3.1.2. SPLAT

A tesztelések során bizonyos példamodellek a `.splat` formátumban voltak elérhetőek. Nincs formális specifikációja[15], egy egyszerű formátum, ami által eltárolt adatok lényegileg a 3.1.1.1-beli `.ply` formátum adatainak felel meg, a SH értékeket elhagyva. A 3.2.1. fejezetben bemutatott szerkesztőprogram támogatja a formátum beolvasását, és `.ply` exportálást, az így kapott `.ply` fájl viszont némiképp eltér bizonyos más fájloktól, mivel nem garantált a paraméterek változatlan sorrendje, illetve az SH értékek nincsenek jelen.

3.1.3. GLTF/GLB

A glTF [12] egy 3D jelenetek és modellek tárolására és átvitelére használt formátum. A Khronos Group adta ki[16], és az egyik legelterjedtebb nyílt formátum erre a célra. 2017-ben, a 2.0 változat megjelenésekor Juan Linietsky, a Godot Engine egyik eredeti fejlesztője a legjobb, és legígéretesebb formátumnak tartotta[17]. 2026-ra a [6] forrás a legelterjedtebb 3D asset átviteli formátumnak titulálta.

A `.gltf` kiterjesztés a szöveges változatot, míg a `.glb` a bináris változatot jelenti, de tartalmilag ugyanaz van bennük. Alapesetben a bináris változat sem tartalmaz masszív tömörítést.

Habár eredetileg szintén nem GS modellekre lett a formátum kitalálva, kiterjesztésekkel viszont könnyen bővíthető: ezt használja ki a [6]-ban bemutatott szabvány is, ami támogatást nyújt a glTF fájlokban történő GS modellek táro-

[15]https://www.polyvia3d.com/formats/splat
[16]https://en.wikipedia.org/wiki/GlTF
[17]https://godotengine.org/article/we-should-all-use-gltf-20-export-3d-assets-game-engines/

lására. Ezt a csúcspontokhoz adott extra adatok segítségével teszi meg, így az extension-t nem támogató megjelenítők is meg tudják jeleníteni a modellt, csak a pontoknak nem lesz kiterjedésük, és az egyéb, extra adatok vesznek el.

Ez alapesetben szintén nagyon nagy fájlméreteket eredményez, mivel nem tömörített formátumról beszélünk. Ezt megoldandó, a [6] kiterjesztés bemutatásakor már folyamatban volt a formátum bővítése, hogy tömörítést is végre lehessen hajtani. Az egyik említett módszer a 3.1.4. fejezetben bemutatott SPZ formátum beágyazása, amelyhez tartozó Pull Request a diplomaterv írásakor is egyeztetés, fejlesztés alatt áll[18].

3.1.4. SPZ

Az `.spz` formátumot a Niantic fejlesztette, nyílt, jól dokumentált szabvány [11]. Agresszívan tömörít, ami a fájlméretnek előnyös, archiválási célra viszont nem. A nyers `.ply` mellett jelenleg ez a másik legelterjedtebb formátum, más-más célra jók.

3.1.5. SOG

A formátumot a [13] cikk vezette be, és az ötlet mögötte az, hogy a tulajdonságokat 2D képként tároljuk el, amire aztán használhatóak a már meglevő 2D képtömörítési módszerek (pl. JPG, PNG). Szintén agresszív tömörítést tud elérni, és kevesebb veszteséget, viszont nem túl elterjedten támogatott.

3.2. Konvertáló programok

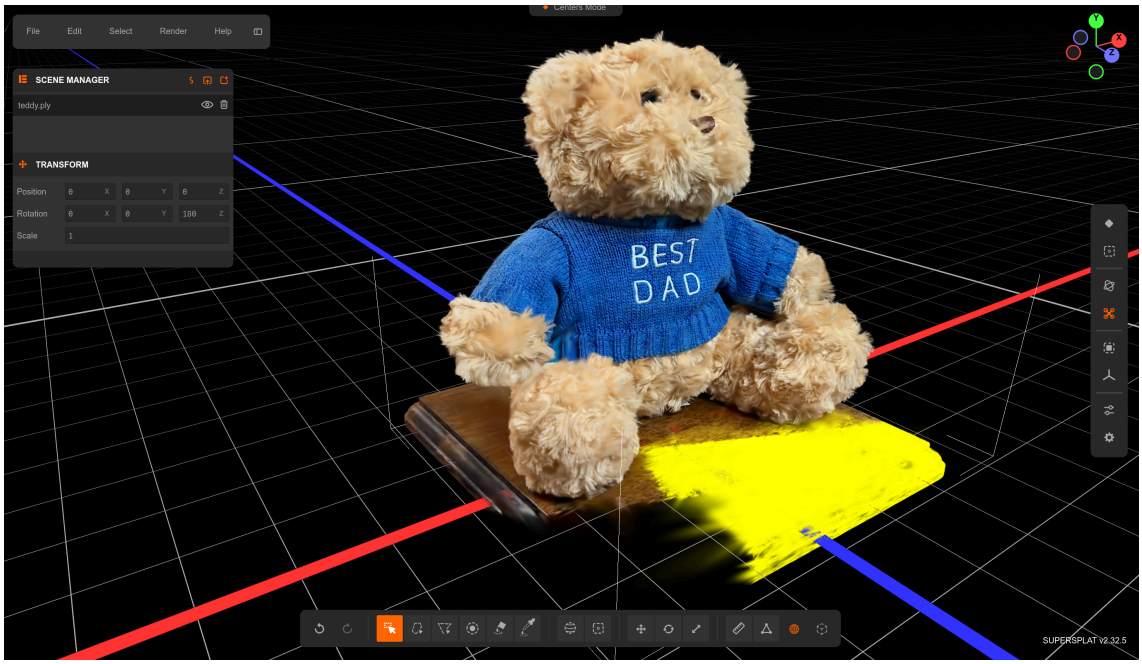
A különböző modellek betöltéséhez és manipulációjához különböző programok használatára is szükség volt. Ezeket itt részletezem.

3.2.1. SuperSplat

A superspl.at egy könnyen használható, online szerkesztő[19], konvertáló és publikációs felület, ami a modellek tisztításához, bizonyos blobok letörléséhez, bizonyos tárgyak környezettől történő elválasztásához kifejezetten hasznos volt számomra is. Be lehet tölteni sokféle fájlformátumot, meg lehet nézni a modellt, bizonyos részeket levágni, vagy éppen több modellt összekombinálni, majd az eredményt különböző formátumokba exportálni.

[18] https://github.com/KhronosGroup/glTF/pull/2531
[19] https://superspl.at/editor

Az 3. ábrán az egyik modell talapzatának levágását illusztrálom. Természetesen pontosan a Z tengely irányába nézve, a tengely ábrázolt magasságában lebegve sokkal egyszerűbb a teljes talapzatot kijelölni, és levágni.

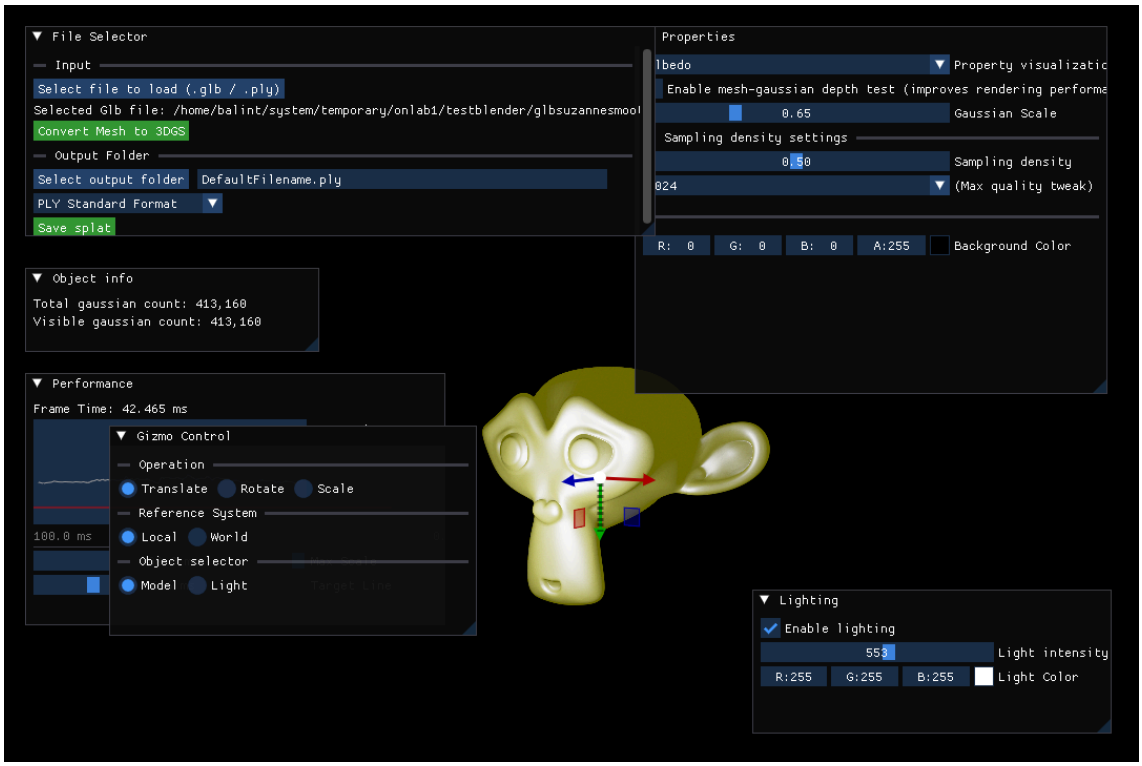


3. ábra: A SuperSplat Editorban a Teddy modell talapzatának levágása

3.2.2. Mesh2Splat

A Mesh2Splat az Electronic Arts GitHub felületén elérhető konvertáló program [3], a fejlesztő diplomatervéből nőtte ki magát. A célja, hogy a szokványos, háromszögekből álló modelleket átalakítsa GS modellekké. A felülete a 4. ábrán látható.

Számunkra a 2.1.2. fejezetben taglalt újravilágításhoz hasznos, szükséges, mivel az exportálás eredményeként a pacáknak (blob-oknak) PBR anyagtulajdonságokat is ad, a 3.1.1.2. fejezetben leírt formátum részeként.



4. ábra: A Mesh2Splat konvertáló program betölti Suzanne-t

3.2.2.1. Linux port

A programot cross-platform keretrendszerben írták meg, C++-t, OpenGL-t és ImGUI-t használ, viszont nem volt sosem tesztelve Linuxon. Így ahhoz, hogy könnyen használni tudjam, bizonyos módosításokat el kellett végezzek a programon. A legfőbb módosítások pár fájlnév nagybetűjének átírása, pár Windows-only include lecserélése, és a fájl tallózásnál az útvonalban egy javítás volt.

A problémáról nyitottam egy issue-t[20], majd a maintainer kérésére egy pull request-et is[21]. Sajnos a változtatás végül nem lett merge-elve, feltételezhetően idő- vagy motivációhiány miatt, habár több embertől érkezett rá pozitív visszajelzés. Később egy másik contributor szintén elvégezte[22] a Linux támogatáshoz szükséges változtatásokat, amik már némiképp mások voltak, mivel a kódbázis is változott, és ez szerencsére merge-elve lett, így a projekt jelenleg már támogatja a Linuxon történő fordítást és futtatást is.

3.2.3. SplatTransform

A [6] által írt glTF GS extension nagyon szépen specifikálva volt, a specifikáció mellett viszont sehogyan sem sikerült találnom példafájlokat, amivel

[20]https://github.com/electronicarts/mesh2splat/issues/2
[21]https://github.com/electronicarts/mesh2splat/pull/7
[22]https://github.com/electronicarts/mesh2splat/pull/22

az implementációt tesztelni tudtam volna. Sajnos a 3.2.1. fejezetben bemutatott SuperSplat weblap nem támogatja, legalábbis egyelőre, a glTF exportot, így szükségem volt egy másik konverziós programra, amivel elő tudtam állítani a szükséges tesztfájlokat.

Így esett a választásom a [14] -nél fellelhető `splat-transform` nevű parancssori programra, ami pont arra jó, hogy GS modelleket lehet vele parancssorból konvertálni.

Az installálás `npm`-en keresztül tűnt a legegyszerűbbnek, a használat pedig szerencsére nem igényelt komplex szintaktikát:

```
splat-transform input.ply output.gltf
```

3.3. Használt programnyelvek ismertetése

A feladat elvégzéséhez a Godot által nyújtott környezetet szükséges használni. Erre sokféle nyelvi opció van, de a legegyértelműbb választás az alábbi.

3.3.1. A GDScript nyelv

A Godot játékmotor alapértelmezetten rendelkezik egy saját nyelvvel, ami a Python-hoz hasonlít leginkább, némi apróbb módosításokkal. Könnyű megismerni, használni, viszont compute-intenzív feladatokra nem a leggyorsabb. Szerencsére a feladat során az ismétlődő, compute-intenzív részt úgyis GPU-oldali kód fogja végezni.

Habár ez egy, a motoron belül futó scriptnyelv, a motor mégis sok funkciót elérhetővé tesz a nyelvből, így például egy vékony absztrakciós rétegen keresztül az alacsony szintű grafikus API-t is, ami általában Godot esetében Vulkan-t jelent. Ezt a RenderingDevice[23] nevű osztály segítségével teszi, és ezt tudjuk használni például a saját uniform buffer-ek, descriptor set-ek, és pipeline-ok létrehozására, menedzselésére.

A nyelv szorosan integrálva van a játékmotorba, saját, jól használható szövegszerkesztővel, és sok egyéb quality-of-life feature-rel.

3.3.2. A GDShader nyelv

Godot-ban vertex és fragment shader-t leggyakrabban `.gdshader`-ként

[23] https://docs.godotengine.org/en/stable/classes/class_renderingdevice.html#renderingdevice

írnak[24], így ezt szükséges használnunk ahhoz, hogyha GPU-oldali kóddal szeretnénk testreszabni valaminek a megjelenését. A motor rendelkezik saját editorral is a nyelvhez. A nyelv leginkább a GLSL ES-hez hasonlít, egy pár különbséggel[25], mint például azzal, hogy egy fájlba kerül a vertex és a fragment shader.

3.3.3. A GLSL nyelv

A `.glsl` nyelv eredetileg az OpenGL-hez készült, a GPU-n történő árnyalás programozásának könnyebbé tétele érdekében[26]. A később megjelent Vulkan API-val is használható, és compute shader-öket is lehet vele írni. Godot-ban a compute shader-ök írására nem hoztak létre egy külön nyelvet, így ezeket közvetlenül GLSL-ben lehet írni[27]. A szöveg szerkesztésére saját szövegszerkesztő szükséges, a motor csak a szintaktikai ellenőrzést, és a fordítást tudja nekünk elvégezni.

3.4. Godot bővítésének lehetőségei

A motor hivatalosan nem támogatja a GS modellek megjelenítését, a pipeline-ja háromszöghálós modellek megjelenítésére lett kitalálva, és optimalizálva. Emiatt fontos kérdés, hogy milyen módon tudunk belenyúlni a pipeline-ba, hogy az általunk kiszámolt, gausszi pacás képet össze tudjuk kombinálni az engine által kiszámolt, háromszöghálós modellek nézetével. Ez nem egy egyértelmű feladat, és többféle módszer is rendelkezésünkre áll, előnyökkel és hátrányokkal.

3.4.1. Megjelenítés teljes képernyős quad-ként

Godot 4.3 előtt az volt a legkézenfekvőbb megoldás a problémára, hogy a teljes képernyőre rajzolunk egy téglalapot, és erre tesszük rá az általunk megjeleníteni kívánt képet. Ez egy működő megoldás, picit nehézkes megfelelően létrehozni a textúrákat, és az erőforrások felszabadítása nem egyértelmű. Ezen kívül nekünk, GDScriptből, magas szintről szükséges minden alkalommal a frissítést meghívni, ami működik, de nem a legszebb megoldás.

Ezt a megoldást implementálta a 2.3.1. fejezetben bemutatott projekt, ami az én munkám kiindulópontját is adta.

[24]https://docs.godotengine.org/en/stable/tutorials/shaders/shader_reference/shading_language.html

[25]https://docs.godotengine.org/en/stable/tutorials/shaders/converting_glsl_to_godot_shaders.html#doc-converting-glsl-to-godot-shaders

[26]https://en.wikipedia.org/wiki/OpenGL_Shading_Language

[27]https://docs.godotengine.org/en/stable/tutorials/shaders/compute_shaders.html

3.4.2. Megjelenítés compositor effect-tel

Godot 4.3-ban megjelent egy módszer, amivel a kompozitort lehet bővíteni, CompositorEffect néven[28]. Ezzel gyönyörűen félbe lehet szakítani a renderelés menetét, és beilleszteni a saját adatainkat a megjelenítéshez. Viszont ennek is vannak limitációi, például újravilágítás esetén nem lehet a Godot deferred shading-jét használni a splat-os modellekhez, így itt az árnyalásra más módszert szükséges alkalmazni. A 3.4.1. fejezetbeli megoldással lehetséges a Godot beépített deferred shading-jét használni.

A 2.3.3. fejezetben bemutatott projekt a 2.3.1. fejezetbeli projektből kiindulás után áttért erre a módszerre, így a kollaboráció során az általam létrehozott kód egy része is ezt a megoldást használja.

[28]https://docs.godotengine.org/en/stable/classes/class_compositoreffect.html

4. Fejezet

Tervezés

Ebben a fejezetben bemutatom a programmal, környezettel szemben támasztott elvárásokat, a megvalósítás érdekében használt architektúrát, és a projekt végrehajtásának menetét.

4.1. Követelmények

Mint minden programmal szemben, ezzel szemben is vannak követelmények támasztva. Ezeket csoportosítani tudjuk funkcionális, és nem funkcionális követelményekre.

4.1.1. Funkcionális

Legfontosabb követelményként a megoldásnak végre kell tudnia hajtani a feladatkiírásban szereplő feladatokat, illetve ki kell tudnia elégíteni a feladatkiírásban explicit nem szereplő, de a belőle következő igényeket.

Ennek alapján a játékmotornak funkcionálnia kell a bővítés után is, látható kell legyen a (potenciálisan több) GS modell a háromszöghálós modellek mellett mélységhelyesen, ezek szükség esetén mind legyenek képesek külön mozogni, akár programozhatóan reagálni a környezetükre, részt venni a fizikában, illetve bizonyos esetben a környező fény, fények által megvilágítódni.

A program legyen képes különböző formátumú GS modelleket beolvasni, legalább a `.ply` és a `.gltf` / `.glb` fájlokat, mivel az előbbi a legelterjedtebb, az utóbbi pedig egy nagy szabványosító szervezet által jött létre, és a feladatkiírás explicit kéri.

4.1.2. Nem funkcionális

A nem funkcionális követelmények közé sorolhatók, mint általában[29], a stabilitás, erőforráshasználat, sebesség, a megjelenítés minősége és a szélesebb hardver-támogatás is. Ezek közül az erőforráshasználat szerepel explicit a feladat-

[29]https://hu.wikipedia.org/wiki/Nem_funkcion%C3%A1lis_k%C3%B6vetelm%C3%A9ny

kiírásban, az instancing vizsgálatával, megvalósításával. A projekt során a többi tulajdonságra is figyelek, amennyiben a scope keretein belül jelentősen fejleszthetők, azt elvégzem. De az elsődleges cél a funkcionalitáson van.

4.2. Architektúra

A 3.4. fejezetben bemutatott bővítési lehetőségtől függően többféle változata is lehetséges az architektúrának, és ezek közül mindkettőnek vannak előnyei és hátrányai, így mindkettővel dolgozok. Ez, és más körülmények miatt sajnos a kódbázis, és az addon különböző verziói más-más állapotban vannak. Ez azért szükséges, mivel például különböző változatait az újravilágításnak más-más verzióban, architektúrával lehet implementálni, így párhuzamosan több változat is szerepel. Ezután az egyéb feature-öket más-más kódbázisban implementálom, és a fontosabbakat portolom a másik kódbázisba is. Ez, attól eltekintve, hogy kényelmetlen, hasznos fejlődési lehetőség számomra. Ezeket az elvégzett tevékenységeket a 5. nagyfejezetben részletesen taglalom.

A kép motorba való beillesztésén kívül az architektúra megegyezik, és a következőből áll. Van egy-egy GLSL shader a rendezésre, ez a [15]-ben megjelölt könyvtár módosított változata, mivel a 2.3.1. fejezetben leírt program is azt használta. Ezután a splatokat további GLSL shaderökkel vetítjük, és rendereljük. Az árnyalók menedzselését egy GDScript oldali szkript végzi, hozza létre a pipeline-t, állítja be a uniformokat, és az egyéb buffereket. Ennek segítésére létezik egy kontextus objektum, ami elabsztrahál bizonyos alacsonyabb szintű műveleteket, hogy a kód olvashatóbb legyen. Ez is GDScriptben íródott. Ez az architektúra túlnyomórészt a 2.3.1. fejezetben leírt programból származik, az én feladatom ennek a bővítése, karbantartása a feladatkiírásban leírt feladatok elvégzése során.

Az egyéb bővítések, mint például a fájl importálásáért, az erőforrás (resource) definíciójáért, illetve a jelenetbe helyezhető node-leszármazottért felelős kódrészletek is mind GDScript-oldaliak. Ezeknek a felelőssége többé-kevésbé jól elhatárolt, és külön-külön fájlokban kapnak helyet. A fájlformátum beolvasása és az erőforrás definíciója és értelmezése már egy kevésbé elhatárolt terület például, ezek a felelősségi körök és célok az egyszerűség függvényében az implementáció közben könnyen meghatározhatóak, illetve módosíthatóak. A tényleges beolvasást, értelmezést végző kódrészlet az egyik objektumból a másikba könnyen átemelhető

úgy, hogy a funkcionalitás ne változzon. Ez igaz az egyéb, nehezen meghatározható felelősségi körökre is.

4.3. Projekt végrehajtása

A projekt végrehajtása során körülbelül a feladatlap sorrendjében megyek végig a funkciókon, nézek utána az implementálás lehetőségeinek, és implementálom őket. Ez a végrehajtási sorrend nem feltétlenül egyezik meg az 5. fejezetben leírt sorrenddel, mivel ott logikus sorrendben szeretném bemutatni az elvégzett munkát, néhol időben egymástól távol elvégzett, de összetartozó feladatokat együtt csoportosítva.

5. Fejezet

Önálló munka bemutatása

Az önlab 1-es GitHub repository-m itt [16] érhető el, a kóddal és a dokumentációval együtt.

Adott a megtekintő program, amely be tud tölteni egy `.ply` kiterjesztésű splat-os modellt, és meg tudja azt jeleníteni a játékmotor keretein belül. Illetve adott példaként az autó-szimuláló program. Adottak továbbá splat-os modellek, amiket lehet használni.

Ennek megfelelően a féléves munka a következő részegységekre osztható:

- Megjeleníteni egyszerre több pontfelhőt
- Ezeket a pontfelhőket függetlenül mozgatni
- Feltölteni pontfelhőnként transzformációs mátrixokat, hogy tetszőlegesen mozoghassanak a modellek
- Ezeket a transzformációkat megfelelően feltölteni minden képkockában, lehetőleg automatizált módon megtalálva a hozzájuk tartozó játékobjektumot, ahonnan a transzformáció származik
- Mélységhelyesen összekombinálni a pontfelhős, és a sima modelleket
- Összevonni a két projektet (gsplat-os, illetve az autós)
- Robusztusabbá tenni a `.ply` fájl-beolvasást, hogy a `.splat`-ból [SuperSplat] (https://superspl.at/editor)-tal konvertált `.ply` fájlokat is képes legyen beolvasni
- Javítani felmerülő problémákat
  - Stuttering: camera és transzformációs mátrix feltöltésének a process priority állítása lett a megoldás
  - Artifactos mélység: a shader kód párhuzamosan tölt be adatokat, és dolgozza fel őket, ehhez viszont szükséges shared bufferek használata

5.1. Egyszerre több pontfelhő megjelenítése

5. ábra: Egyszerre látható több pontfelhős modell

a 5. ábrán egyszerre látszik a mozdony és a környezete, amik egy fájlból származnak, illetve a bonsai fa teteje, ami egy másik fájlból származik. A transzformáció itt még nem működik, így nem lehet mozgatni sem a két modellt egymáshoz képest, az majd a 5.2. fejezet témája lesz.

A megjelenítő egy pontfelhő megjelenítésére volt felkészítve, nekünk viszont egy játékban jórészt több pontfelhőt is célszerű megjeleníteni: például az autó teste és a 4 kereke már eleve 5 külön objektum. Ehhez azt találtam ki, hogy a CPU-n összeuniózom a pontfelhőket, az adataikat, viszont elmentem azt is, hogy mely indexeknél vannak a határok. Ezután a GPU-ra feltöltéskor kihasználok egy float-ot, ami eddig padding célt szolgált, és minden feltöltött splat esetén egy ID-t is továbbítok, ami megmondja, hogy hányadik objektumhoz tartozik.

```
class_name PlyFile extends Resource

var size : int
var vertices : PackedFloat32Array
var properties : Array[StringName]
var split : Array[int] # indices where new objects start

...

static func merge(pc1 : PlyFile, pc2 : PlyFile) -> PlyFile:
  var merged := PlyFile.new()
  merged.size = pc1.size + pc2.size
  assert(pc1.properties.hash() == pc2.properties.hash())
  merged.properties = pc1.properties
```

```
  merged.vertices = PackedFloat32Array(pc1.vertices)
  merged.vertices.append_array(pc2.vertices)
  merged.split.append_array(pc1.split)
  merged.split.append(pc1.vertices.size())
  for s in pc2.split:
    merged.split.append(s + pc1.vertices.size())
  return merged

...

static func load_gaussian_splats(point_cloud : PlyFile, stride : int,
    device : RenderingDevice, buffer : RID, should_terminate_reference :
Array[bool],
    num_points_loaded : Array[int], callback : Callable):
...
      ### Opacity
      points[b+6+4] = 1.0 / (1.0 + exp(-p[v+54]))

      ### ID for differenciating between objects
      points[b+11] = 0
      for k in point_cloud.split:
        if v >= k:
          points[b+11] += 1
...
```

5.2. Pontfelhők transzformációja

6. ábra: A bonsai fa transzformálva van, míg a vonat helyben marad

a 6. ábrán … és … …

Miután a GPU-n már lehet tudni, hogy melyik splat melyik objektumhoz tartozik, így lehetséges objektumonként más transzformációt alkalmazni a splatokra. Ehhez létre kellett hozni, és fel kell tölteni egy új uniform buffert:

```
const MAX_OBJECT_COUNT := 16 # number of gsplat object transforms, same
as in gsplat_projection.glsl
var object_transforms : Array[Transform3D]
...

  descriptors['transforms'] =
context.create_uniform_buffer(16*MAX_OBJECT_COUNT*4)

...

  context.device.buffer_update(descriptors['transforms'].rid,
```

```
        0, 16*MAX_OBJECT_COUNT*4, get_transforms())

...

func update_object_transforms(transforms: Array[Transform3D]) -> void:
  object_transforms = transforms

func get_transforms() -> PackedByteArray:
  var fbuf := PackedFloat32Array()
  assert(len(object_transforms) <= MAX_OBJECT_COUNT)
  var t := []
  for i in object_transforms:
    t.append(Projection(i))
  for i in (MAX_OBJECT_COUNT - len(object_transforms)):
    t.append(Projection.IDENTITY)
  for i in MAX_OBJECT_COUNT:
    fbuf.append_array([ t[i].x[0], t[i].x[1], -t[i].x[2], -t[i].x[3],
              t[i].y[0], t[i].y[1], -t[i].y[2], -t[i].y[3],
              -t[i].z[0], -t[i].z[1], t[i].z[2], t[i].z[3],
              -t[i].w[0], -t[i].w[1], t[i].w[2], t[i].w[3]])
  var bytebuf := PackedByteArray()
  bytebuf.resize(4 * fbuf.size())
  bytebuf.fill(0)
  for i in range(len(fbuf)):
    bytebuf.encode_float(i*4, fbuf[i])
  return bytebuf
```

Amit pedig a shaderben fel tudunk használni:

```
// same as in gaussian_splatting_rasterizer.gd
#define MAX_OBJECT_COUNT 16

...

struct Splat {
  vec3 position;
  float time;
  float covariance[6]; // Contains top triangle of symmetric matrix
  float opacity;
  float id;
  float sh_coefficients[16*3]; // Spherical harmonic coefficients in
increasing order
```

```
};

...

layout (std140, set = 0, binding = 7) restrict uniform Transforms {
  mat4 transforms[MAX_OBJECT_COUNT];
};

...

  // --- FRUSTUM CULLING ---
  vec3 splat_pos = splat.position*model_scale;
  vec4 view_pos = view_matrix * transforms[int(splat.id + 0.5)] *
vec4(splat_pos, 1.0);
  vec4 clip_pos = projection_matrix * view_pos;

  ...

  mat3 curr_transform = mat3(transforms[int(splat.id + 0.5)]);
  mat3 cov_mx = curr_transform * DECODE_COVARIANCE(splat.covariance) *
transpose(curr_transform);
  const vec3 covariance = project_covariance(cov_mx, splat_scale,
view_pos.xyz, dims);

...
```

5.2.1. Kiterjedés irányainak transzformációja

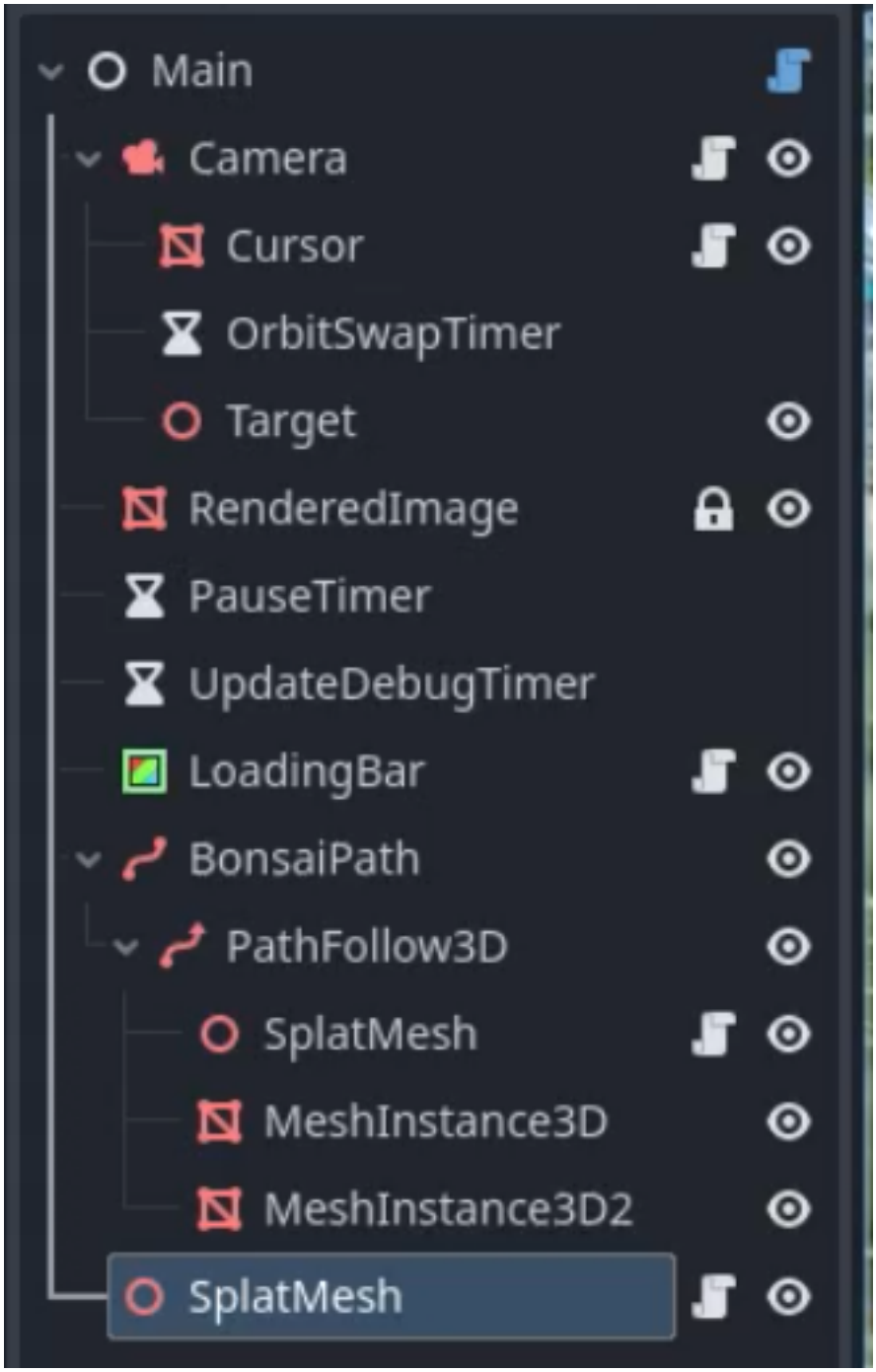
itt vagy lentebb ki lehetne emelni azt a részt amit a kínai csávó is átvett; megreferálni a lentebbi fejezetet? meg előtte leírni hogy miről is van szó

5.2.2. Gömbi harmonikusok transzformációjának megjavítása

Issue fix-szel a kínai csávónak (megreferálni a fejezetet lentebb?)

ide egy screenshot a különbségről, pl az issue-ból !?

5.3. Automata objektum-megtalálás



7. ábra: Egy lehetséges SceneTree, ahol a SplatMesh-eket meg kell találni

TODO ábra legyen kisebb? vagy legalábbis kevésbé elmosott/pixeles.

7. ábrán látható egy …, a 6. ábrának közel megfeleltethető: van egy, splineon mozgó GS objektum, mellette kettő sima háromszöghálós modell ugyanazzal a transzformmal (hogy lehessen ellenőrizni a korrekt pozíciót), és van egy másik, közvetlenül a gyökér gyerekeként, így ő nincs transzformálva, de legalábbis dina-

mikusan biztos nem. Ennek a két objektumnak a globális transzformját kell összegyűjteni, hogy a pontfelhőket a megfelelő pozícióra lehessen helyezni.

Jelenleg nem támogatott új objektumok létrehozása illetve eltüntetése, láthatóvá illetve láthatatlanná tétele futásidőben. Támogatott viszont az, hogy egy, a Godot Node-ok között is megtalálható objektumot a SceneTree-hez hozzáadva automatán be legyen töltve a splat-os objektum, és a megfelelő Node transzformációja legyen rá hattatva, figyelembe véve a szülő node-ok transzformációját is, így lehetséges valós időben mozgatni, transzformálni őket ugyanúgy, mint a sima modelleket.

A következő script a `class_name` segítségével meg fog jelenni az editorban, mint hozzáadható Node, és lesz egy állítható paramétere, a `ply_file`. Ide be lehet állítani a megfelelő path-t.

```
class_name SplatMesh extends Node3D

@export var ply_file: String

func is_splat_mesh():
  pass
```

Ezután a gyökérnode induláskor összeszedi az objektumokat, és eltárolja őket, hogy a transzformációikat el tudja küldeni:

```
var splat_meshes : Array[SplatMesh] = []

func _ready() -> void:
  find_by_method(self, StringName("is_splat_mesh"), splat_meshes)
  assert(len(splat_meshes) <=
GaussianSplattingRasterizer.MAX_OBJECT_COUNT)
  var splat_filenames := []
  for m in splat_meshes:
    splat_filenames.append(m.ply_file)

  init_rasterizer(splat_filenames)
    ...

# source: https://forum.godotengine.org/t/how-do-you-get-all-nodes-of-a-
certain-class/9143
func find_by_method(node: Node, method_name : StringName, result : Array)
-> void:
```

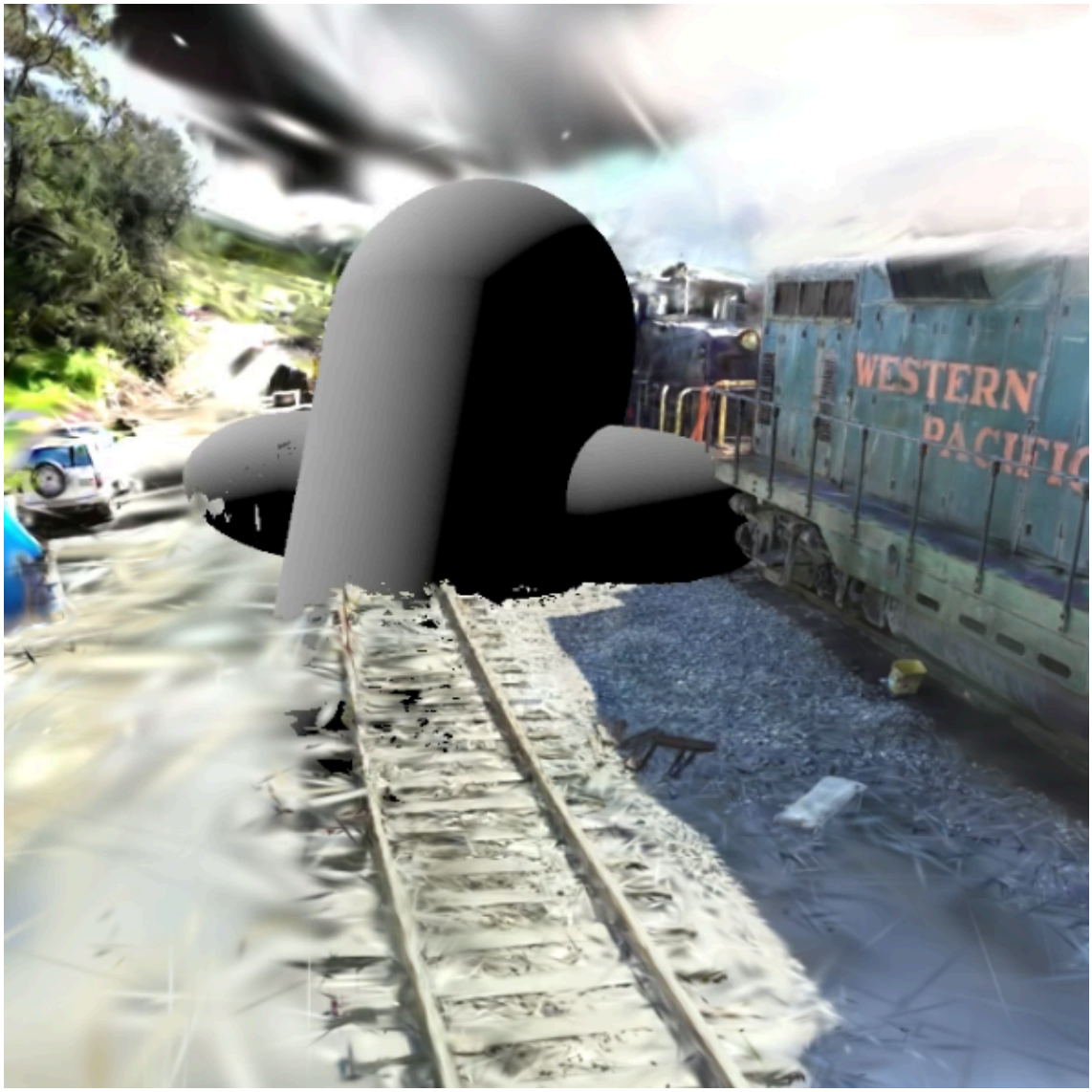
```
  if node.has_method(method_name) and node.is_visible_in_tree():
    result.push_back(node)
  for child in node.get_children():
    find_by_method(child, method_name, result)
```

Az összeszedéshez biztosan van jobb módszer, mint a tagfüggvénynév alapján való keresés, de én limitált keresés során nem találtam, és ez működik. Típuslekérdezés csak beépített típusokra ad eltérő eredményt.

```
func _process(delta: float) -> void:
    ...
  var splat_transforms : Array[Transform3D] = []
  for m in splat_meshes:
    splat_transforms.append(m.global_transform)
  rasterizer.update_object_transforms(splat_transforms)
```

Fontos megemlíteni a szálakat, ugyanis a rasterizer függvényeit alapvetően a render thread-en hívjuk, ez az objektum-transzformációk feltöltése viszont, a kameratranszformációk lekérdezéséhez hasonlóan a „sima” thread-en történik.

5.4. Mélységhelyes összekombinálás



8. ábra: A mélység összekombinálásának tesztelése a vonatos jelenettel és beépített kapszulákkal

a 8. ábrán látható …

Ahhoz, hogy a képen össze lehessen fésülni a pontfelhős, és a sima megjelenítést, amelyek teljesen külön pipeline-on futnak, szükséges a pontfelhős megjelenítésnél is, a végső renderelt képen egy pixelenkénti mélységérték szerzése. Ennek segítségével legalább pixelenként el lehet dönteni, hogy a két renderelt kép közül melyik nyerjen. A sima pipeline-nál már eléri a játékmotor a mélységet, nekünk a splat-os megjelenítésnél kell ezt elérhetővé tennünk:

Projection:

```
data.pos_z = (ndc_pos.z + 1.0)*0.5;
```

Render:

```
shared float[WORKGROUP_SIZE] pos_z_tile;

...

    for (uint i = 0; i < num_iterations && shared_t > MIN_FACTOR; ++i) {
    ...
        for (uint j = 0; j < chunk_size && t > MIN_ALPHA; ++j) {
            ...
            float pos_z = pos_z_tile[j];
            ...
            if (alpha > 0.2) {
                weighted_depth += (1.0 - pos_z) * alpha * t;
                total_weight += alpha * t;
            }
            ...
            t *= (1.0 - alpha);
        }
        ...
    }
    vec3 heatmap_color = mix(vec3(0,0,1), vec3(1,0.2,0.2),
num_splats*5e-4) * (1.0 - t) * heatmap_factor;
    float final_depth = total_weight > 0.0 ? (weighted_depth /
total_weight) : 0.0;
  imageStore(rasterized_image, ivec2(image_pos), vec4(blended_color +
heatmap_color, final_depth));
  ...
```

Main spatial shader:

```
void fragment() {
  ALBEDO = srgb_to_linear(texture(render_texture, SCREEN_UV).rgb);
  DEPTH = texture(render_texture, SCREEN_UV).a;
}
```

5.5. Addon készítése

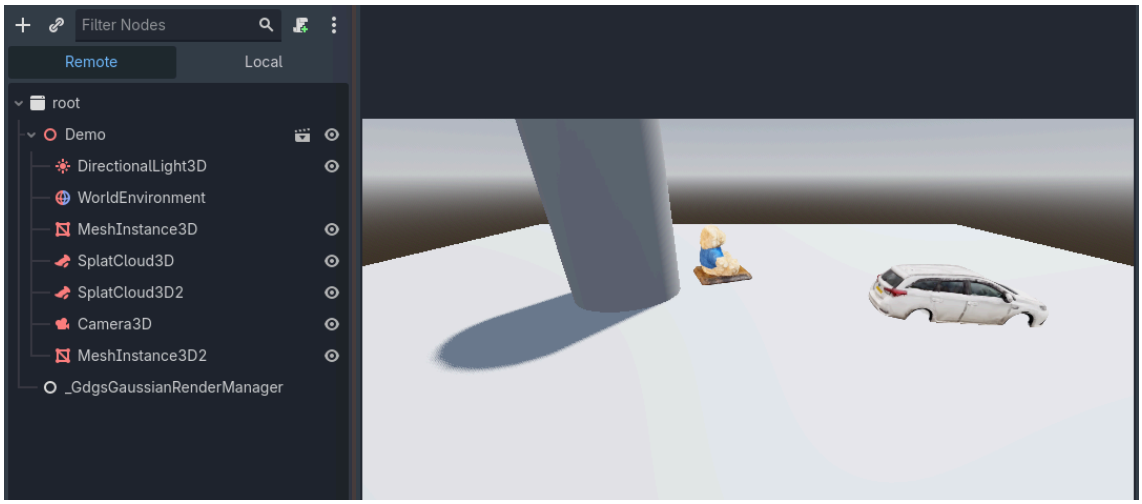
leírni hogy milyen fájlokat kellett hova tenni át, autoload, registration script, stb

5.6. Pull vagy Push modell

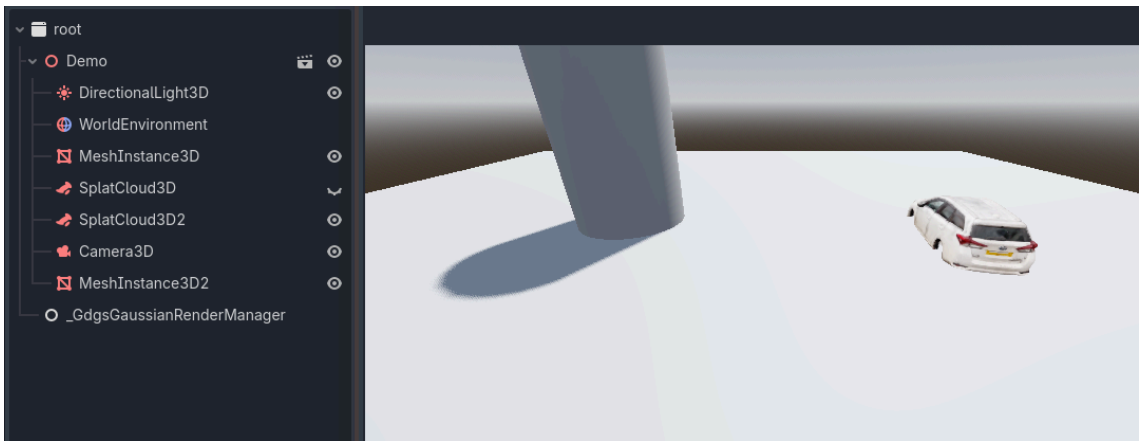
RenderedImage pull vs SplatMesh push

Az eredeti megoldás szerint a RenderedImage válogatta össze a jelenetből a kirajzolandó pontfelhő node-okat, és a transzformjaikat. Ezzel az új megoldással a pontfelhő node-ok maguk regisztrálhatnak, és jelentkezhetnek le a kirajzolásról.

Ez azt is könnyűvé tette, hogy amennyiben a node láthatósága változik, akkor leregisztráljon, és ezzel eltűnjön ténylegesen a jelenetből. Ez része annak a törekvésnek, hogy a pontfelhők az editorba, illetve a játékba is jobban legyenek integrálva, azaz az editorba épített feature-ök (mint pl. a láthatóság) úgy működjenek rajtuk, mint ahogy a felhasználók várják. A nem látható pontfelhő az 9. ábrához képest a 10. ábrán láthatóan nem látható.



9. ábra: Látható a plüssmaci



10. ábra: Nem látható a plüssmaci

Hozzá kell tenni, hogy ez a megoldás még nem volt a legcélravezetőbb: minden láthatóságváltoztatáskor a teljes éppen aktuálisan látható splatos adatot feltöltöttük a GPU-ra. Ez a 5.6.2. fejezetben került javításra.

5.6.1. Dinamikus láthatóság

a fentebbi fejezetből kiszedni az ide tartozó részt

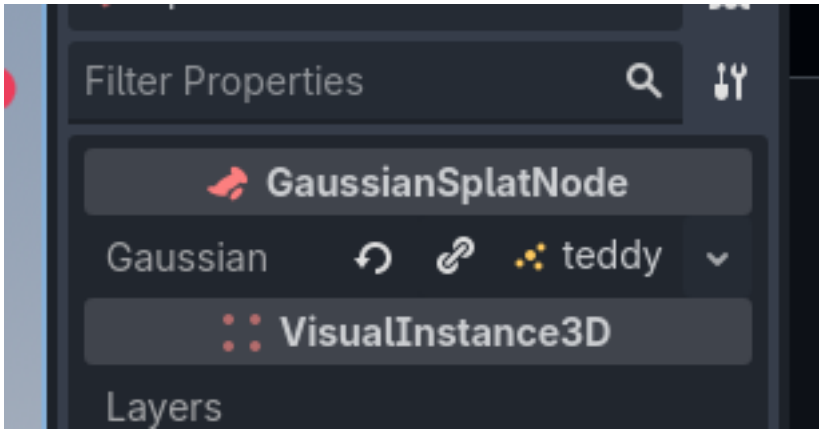
5.6.2. Kifinomultabb dinamikus láthatóság

A PR készítése közben a korábbi dinamikus láthatóság problémájából kiindulva biztosan úgy akartam újból megoldani a problémát, hogy láthatóság változtatásakor ne legyen szükség az összes splat adat újbóli feltöltésére. Ez elég könnyen meg is volt oldható úgy, hogy a transzformáció mellé (ami amúgy is minden mozdulatnál újra feltöltődik) pakoltam egy igaz-hamis értéket, ami alapján a shader el tudja dobni a hozzá tartozó splatokat, ezzel megvalósítva a nemláthatóságot.

5.7. Elsőosztályú polgárok

További editorintegrációhoz egyéb apróságokat is elvégeztem, ilyen a 11. ábrán látható ikonok készítése a splat-os node-okhoz, illetve resource-okhoz, a betöltés és a tárolt adat szétcsatolása, és ezzel az editor automatikusan be is tudja tölteni az eddig számára ismeretlen file-t egy Godot resource-á. Ezzel például exportált buildekhez is automatikusan hozzá tudja tenni ezt a fájlt, és ott már nem kell a betöltést lefuttatnia.

Az 1. táblázatnál láthatóak a node-hoz, illetve a resource-hoz társított ikonok szöveges leírásai is. Ezzel tudom tesztelni a Typst táblázat captionjeit, illetve referenciáit.



11. ábra: A Node és a Resource saját ikonokat kaptak

GaussianSplatNode	GaussianResource
Piros, átfedő pöttyök	Sárga, kisebb sugarú pöttyök

1. táblázat: Táblázat teszt

5.7.1. Editor ikonok

ide áthozni / kijjebb fejteni a fentebbi szöveg tartalmát

5.7.2. Betöltés, és tárolt adat szétcsatolása

meg ide

5.8. AMD bug kijavítása

A tesztelések során figyelmes lettem egy AMD hardveren történő bug-ra, ezt a VulkanRadixSort könyvtárra vezettem vissza, és az eltérő subgroup méret okozhatta. A hibát én is kijavítottam, és láttam azt is, hogy upstream is kijavították már, csak az általam használt eredeti megjelenítő nem frissítette a dependency-t. Ezt jeleztem az ő repositoryjukban is, a megfelelő fixet megjelölve [17]. A linkelt issue-ban találhatóak a további linkek a kapcsolódó fixekre.

5.9. CompositorEffect-es megoldás vizsgálata

Biztos ez jó referencia? nem a fentebbi, bevezető fejezetet kéne?

Godot 4.4-től már lehetőség van a rendering pipeline-ba jobban belenyúlni, CompositorEffect-eket írva. Az eddigi, eredeti megjelenítő egy QuadMesh-t jelenített meg egy MeshInstance3D-vel, ami lényegében mindig az igazi kamera előtt lebegett, így tudta a pontfelhőket megjeleníteni. Én még önlab alatt megcsináltam azt hozzá, hogy az átlátszóság alapján összeollózza az eredeti, sima raszteres kamera által adott képpel, ez viszont több helyen látszott, hogy egy elég összetákolt megoldás volt. Manuálisan kellett meghívni a saját renderelésünket, de csak akkor hogyha tényleg szükségünk volt a képre, különben elkezdett szaggatni az editor. A használt textúrák felszabadításának működése nem világos, illetve ablak átméretezésekor is le kell kezeljünk speciális dolgokat, ami nem egyértelmű, hogy teljes körűen sikerül.

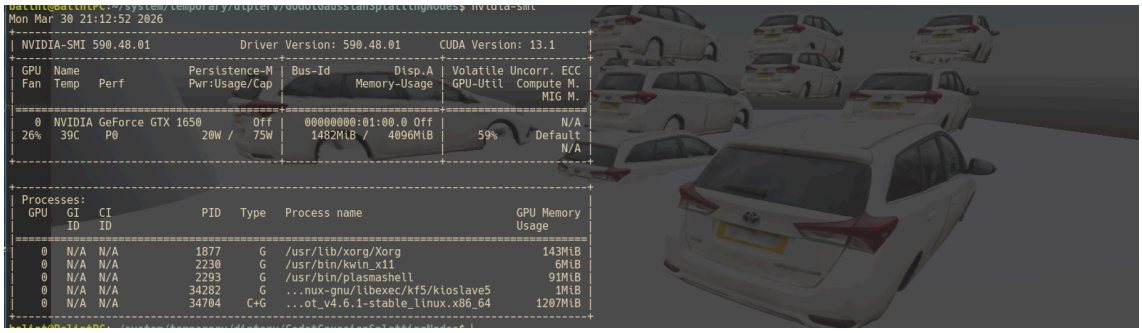
5.10. Instancing

Hogyha már itt voltam a feltöltésnél, úgy láttam, hogy a feladatkiírásban is szereplő instancinget is akkor a legmegfelelőbb implementálni. Ennél a splat-os modelleket kell különválasztani az objektumoktól.

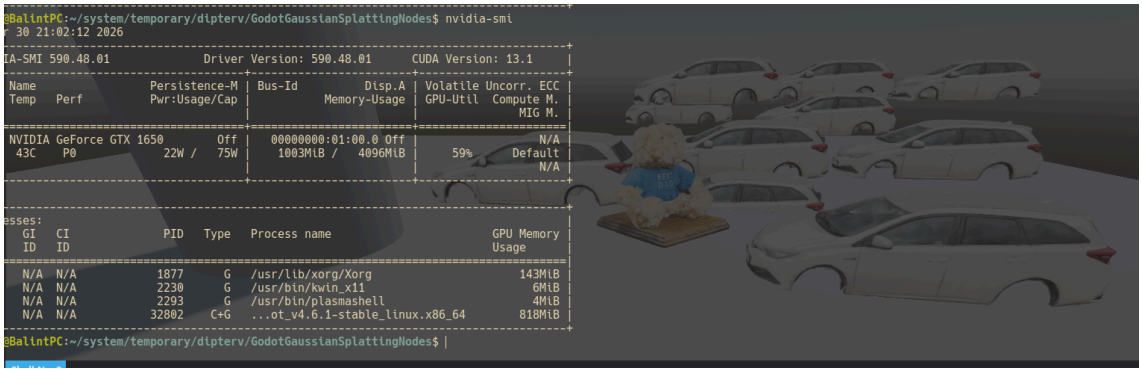
Eddig minden splat egy extra ID-t kapott, és az ID-edik transzformációs mátrixszal lett eltranszformálva, így megoldva azt, hogy az egyes objektumok máshogyan mozogjanak. Az optimalizációs lehetőség ott jön be, hogyha 1 db kocsi

helyett 10 db van a jelenetben, akkor a hozzá tartozó százezer-millió nagyságrendű splat adatait nem szükséges 10-szer tárolni a GPU-n, hiszen ezek ugyanazok.

A megoldás pedig nem nagyon nehéz: az ID buffer legyen kétszer akkora, és minden minden párban az egyik ID mondja meg a transzformációs mátrixot, a másik ID pedig a splatot. Így lehet egy modellnek több transzformációs mátrixa is, csak az ID bufferben kell kétszer benne lennie az összes splat ID-jének. Ez az `nvidia-smi` parancs használatával láthatóan jelentősen javította a memóriahasználatot: a 12. ábrán 1207MiB-t használt az instancing előtti állapot, míg az 13. ábrán 818MiB-t, a képeken látható 10-es nagyságrendű autómennyiséggel.



12. ábra: Instancing előtti memóriahasználat



13. ábra: Instancing utáni memóriahasználat

5.10.1. Instancing hatásának mérése

instancing mérése felhasznált memóriával

ez az instancing mellé menjen

5.10.2. Pull Request

5.11. Újravilágíthatóság

A relightolhatóság egy komplex téma, de egy játékmotor esetén nagyon fontos. Alapból a gaussian splates modellek gyakran rendelkeznek gömbi harmonikusokkal kódolt nézeti irány-függő színekkel, splatonként. Ezzel lehet egyszerű

tükröződéseket, csillanásokat szimulálni, és a modell tud színt változtatni annak függvényében, hogy melyik irányból nézzük.

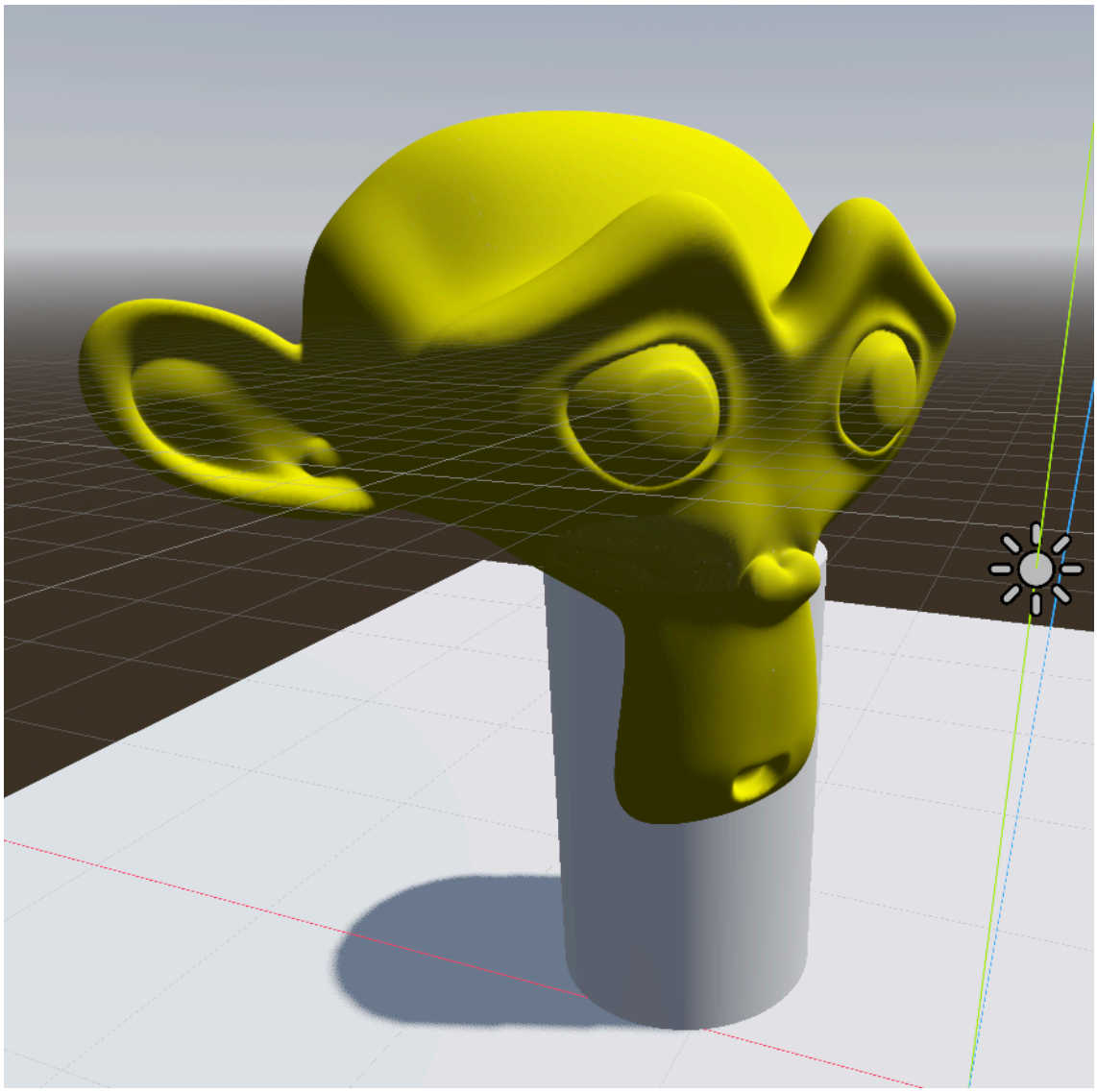
Ezzel szemben egy relightolható / újraszínezhető modell nem nézeti irányfüggő színeket tárol magában splatonként, hanem valamilyen fajta anyagtulajdonságokat, például PBR (roughness, metallic, normal stb.) értékeket. Ennek segítségével nem csak a nézeti iránytól függhet a szín, hanem a teljes, komplex jelenettől, kiemelt hangsúlyt fektetve a fényforrás(ok)ra. Ez egy játékban kifejezetten fontos, például ha egy splat-es modell irányt változtat, akkor a nap másik oldalát süti. Ennek megfelelően a színei is teljesen megváltoznak, amit leginkább ilyen módszerekkel lehet megvalósítani.

A lehetőségek felmérésekor többféle megoldás is célravezetőnek tűnt, mindkettő a saját erősségeivel és gyengeségeivel. Végül arra jutottam, hogy mindkettőt megvalósítom, összehasonlítási céllal.

5.11.1. Saját fényszámítással

Az első lehetőség az, hogy mi adjuk át manuálisan a beszámítandó fényforrásokat, és mi számoljuk a színeket a megadott paraméterekből. A prototípust úgy csináltam, hogy egy irányfényforrást tudunk átadni. Ennek a hátránya a manuális fényforrás-összegyűjtés, illetve a shading eltérő lehet a többi, engine által shadelt mesh-étől. Hatalmas előnye viszont, hogy működik a 5.9. fejezetben tagalt CompositorEffect-es megoldással.

A 14. ábrán is látható, hogy a megvilágítás működik, de a 15. ábrához képest azért alulmarad a többi megvilágítással való egyezőségben. Ezen az ambient light állításával némiképp szerintem lehetne segíteni, de ugyanolyan jó sosem lesz.



14. ábra: Suzanne, a Blender majom megvilágítva általunk

5.11.2. A Godot fényszámításával

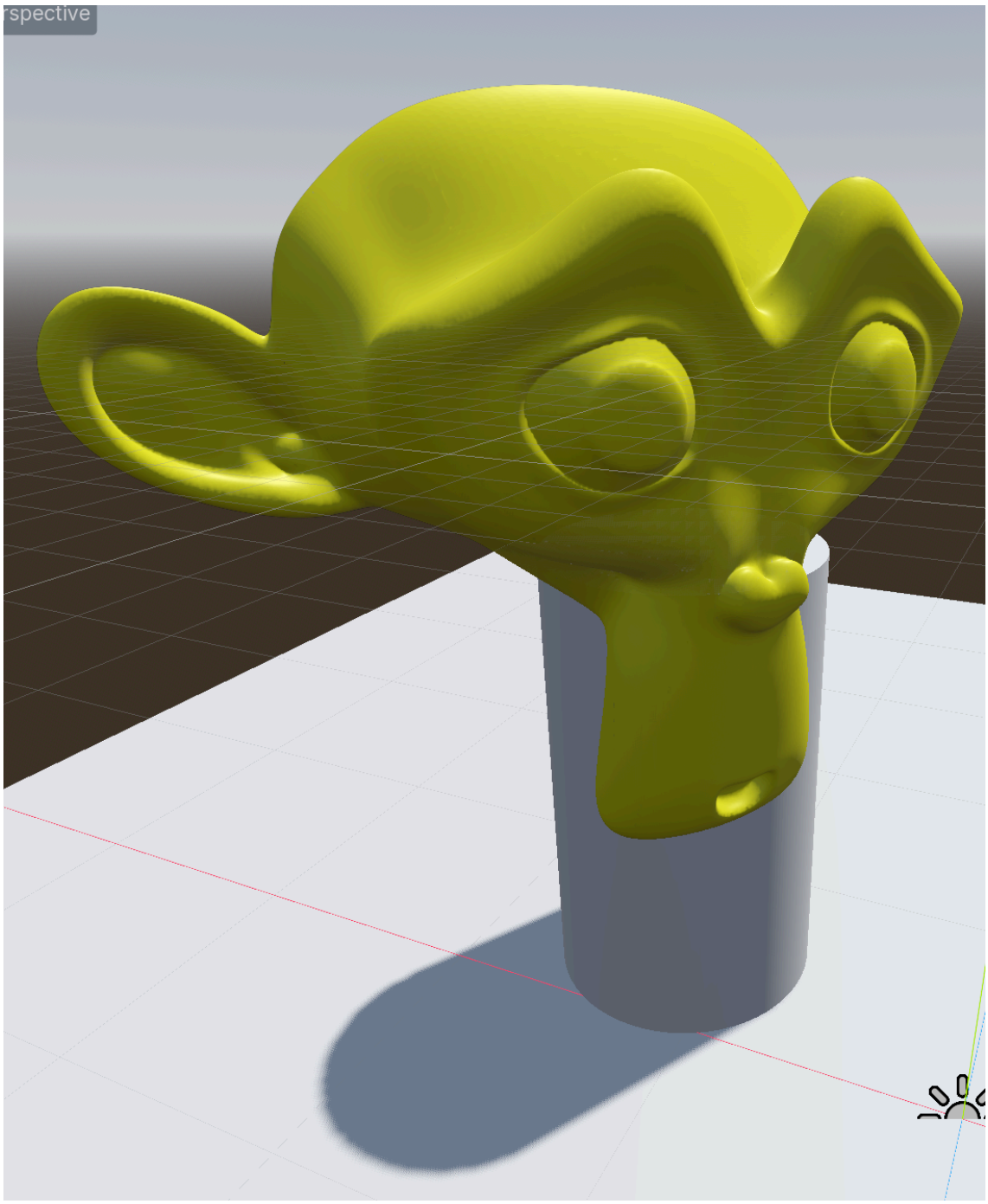
A Godot fényszámításának használatakor viszont elő kellett venni az eredeti, Fullscreen quad-os megoldást, mivel a Godot-nak ezen kereszül tudom átadni a PBR értékeket (egy textúrában), ami alapján ő fogja elvégezni a relightolást, a belül tárolt, gyorsítótárazott, stb. fények alapján. Így ez automatikusan tudna működni pontfényforrásokra és spotlightokra is (csak még egy bug miatt nem tud, lásd A.), amint azt a 16. ábra is mutatja.

Még egy nagyon komoly hátránya ennek a megoldásnak, hogy ez leginkább egyáltalán nem tud félig átlátszó splat-okat kezelni (ami a gaussian splattingnél egy nagyon fontos tulajdonság), mivel a Godot-nak pixelenként csak egy színt, és PBR material tulajdonságokat lehet átadni. Az pedig nem túlzottan megoldható,

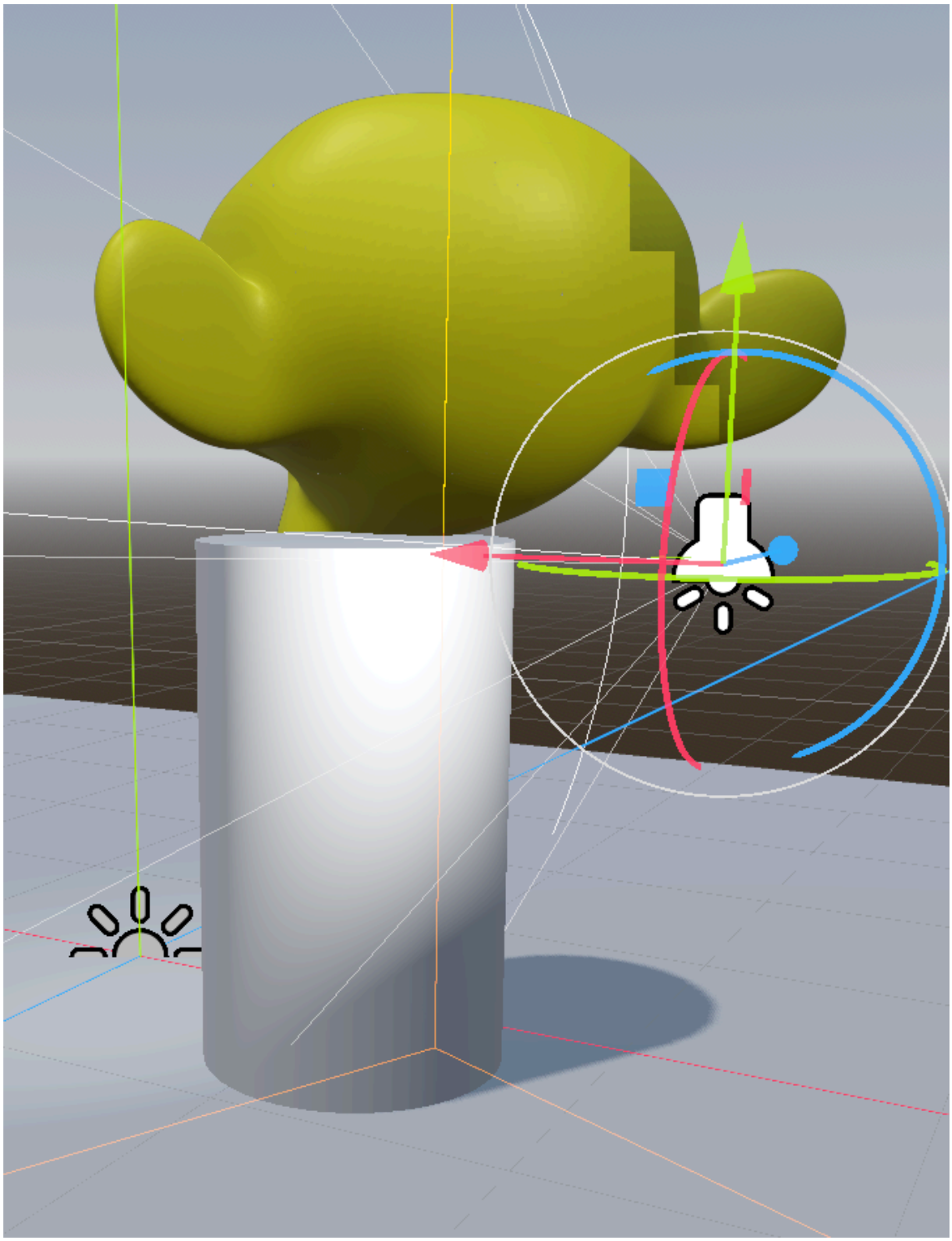
hogy több félig átlátszó splat anyagtulajdonságait átlagoljuk, ezt adjuk át a Godotnak, és továbbra is realisztikus / használható eredményt kapjunk ebből.

Megjegyzendő, hogy a Mesh2Splat-tal generált modelleknél viszont nem fontos az, hogy az átlátszóságot jól kezeljük, mivel ezek a modellek jellemzően nem sok átlátszóságot tartalmaznak, szimpla modelleknél legalábbis.

Ezen kívül viszont, alapból a 15. ábra jóval jobban belepasszol a jelenetbe, mint a 14. ábra.



15. ábra: Suzanne, a Blender majom megvilágítva Godot által



16. ábra: Suzanne, a Blender majom megvilágítva Godot által egy spotlight-tal is (még bugos)

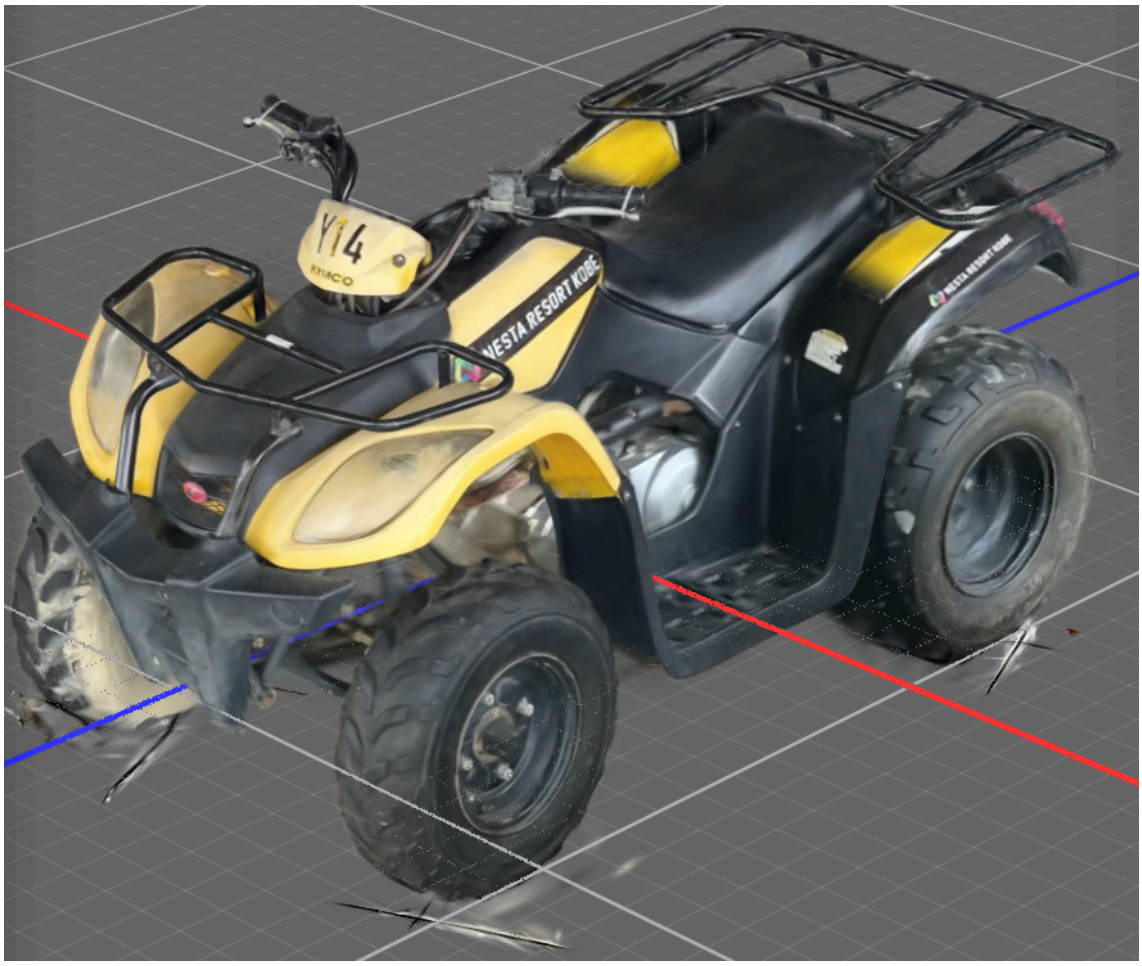
5.11.2.1. Javítás nem irányfény esetére

lepedő, asszem

meg screenshotot csinálni hozzá, magyarázatot, esetleg megemlíteni a limitációkat, bugos széleket, az is növeli az oldalszámot

5.12. Fájlformátumok implementálása

5.12.1. SPLAT adatfájl adatainak beolvasása



17. ábra: Egy `.splat` formátumú buggy modell a SuperSplat editorban

A 17. ábrán látható a példamodell, amit be szerettem volna tölteni. A konverziót `.ply` formává a 3.2.1. fejezetben bemutatott SuperSplat editor elvégzi, viszont a pontos property-k, illetve a sorrendjük nem egyezik meg a korábban megszokottal.

esetleg a teljes konkrét header-öket leírni ide? hogy hogy néz ki módosítás nélkül. Ez szerepel az eredeti, uncut screenshoton pl az önlab1 doksiban

A bemeneti adatformátum a `.ply`, illetve ennek egy speciális esete. Eredetileg a megjelenítő projekt csak olyan modelleket tudott megjeleníteni, amiknek a 62 property-je mind szerepel, és egy konkrét sorrendben van:

```
x, y, z, nx, ny, nz, f_dc_0, f_dc_1, f_dc_2, f_rest_0, f_rest_1,
f_rest_2, f_rest_3, f_rest_4, f_rest_5, f_rest_6, f_rest_7, f_rest_8,
f_rest_9, f_rest_10, f_rest_11, f_rest_12, f_rest_13, f_rest_14,
f_rest_15, f_rest_16, f_rest_17, f_rest_18, f_rest_19, f_rest_20,
```

```
f_rest_21, f_rest_22, f_rest_23, f_rest_24, f_rest_25, f_rest_26,
f_rest_27, f_rest_28, f_rest_29, f_rest_30, f_rest_31, f_rest_32,
f_rest_33, f_rest_34, f_rest_35, f_rest_36, f_rest_37, f_rest_38,
f_rest_39, f_rest_40, f_rest_41, f_rest_42, f_rest_43, f_rest_44,
opacity, scale_0, scale_1, scale_2, rot_0, rot_1, rot_2, rot_3
```

Ez akkor ütközött problémába, amikor egy másik forrásból származó modellt szerettem volna megjeleníteni, ami `.splat` formátumban volt. Addig nem probléma, hogy az online [Supersplat](https://superspl.at/editor) segítségével át lehet konvertálni ezt `.ply` fájllá, viszont ebből a modellből hiányoztak bizonyos property-k, 14 volt összesen, és a meglevők sem megfelelő sorrendben voltak:

```
x, y, z, opacity, rot_0, rot_1, rot_2, rot_3, f_dc_0, f_dc_1, f_dc_2,
scale_0, scale_1, scale_2
```

Így módosítottam a beolvasó részt, hogy 0-ként olvassa a hiányzó adatokat, illetve a megfelelő pozícióba tegye a property-ket. Ez azért működik, mert ami hiányzik, az a 45 db spherical harmonikusokhoz szükséges érték, amiket lehet 0-ra inicializálni, illetve a normálvektor, amit pedig nem használ fel a beolvasó.

Példareferencia egy kódrészletre: a parse függvény az 1. kódrészleten látható.

```
func parse(path : String) -> void:
  var file := FileAccess.open(path, FileAccess.READ)
  var line := file.get_line().split(' ')
  while not line[0] == 'end_header':
    line = file.get_line().split(' ')
    match line[0]:
      'format':   file.big_endian = line[1] == 'binary_big_endian'
      'element':  size = int(line[2])
      'property': properties.push_back(line[2])
  vertices = file.get_buffer(size*len(properties) * 4).to_float32_array()
  if properties.hash() != DEFAULT_PROPERTIES.hash():
    var prop_inverse := {}
    for i in properties.size():
      prop_inverse[properties[i]] = i
    var new_vertices := PackedFloat32Array()
    new_vertices.resize(size*DEFAULT_PROP_CNT)
    for i in size:
      for pi in DEFAULT_PROP_CNT:
        new_vertices[i * DEFAULT_PROP_CNT + pi] =
                    vertices[i * len(properties) +
prop_inverse[DEFAULT_PROPERTIES[pi]]]
                    if DEFAULT_PROPERTIES[pi] in prop_inverse else 0
    properties = DEFAULT_PROPERTIES.duplicate()
    vertices = new_vertices
```

1. kódrészlet: Példa egy kódrészlet számozására

5.12.2. GLTF adatfájl adatainak beolvasása

sh fix a beolvasásnál, PR a kínai csávónak (link a lentebbi fejezetre)

linkelni a fájlformátumra ami fentebb van bemutatva, illetve leírni/megindokolni? hogy nem a tömörített, hanem a sima változat parszolását oldottam meg

5.13. Kollaboráció a másik implementációval

leírni a kezdeti kollab issue-t [10] leírni hogy mit hol implementáltam, fejezetszámmal hivatkozva rá vissza

megjelölni, hogy még mit vett át tőlem (amd bug fix, covarianciamátrix transzformációja a kiterjedéshez)

Ezek miatt felkerült a teendők listájára a CompositorEffect-es megoldásnak a vizsgálata is. Közben a félév során egy kínai implementáció is előkerült itt: [4]. Mérlegelve a szituációt arra jutottam, hogy nem érdemes mindkettőnknek teljesen

külön dolgozni a saját programján, így átfutottam az ő kódját, és mivel mindkét projekt (és az a projekt is, amelyet mindketten használtunk: [17]) MIT licensz alatt elérhető, unióztam a fontosabb feature-jeinket. A kommunikáció felvételekor leírtam 1-2 dolgot, ennek eredménye lett, hogy ő is kijavította az AMD bugot, illetve valamilyen kovarianciás matekot, amitől a splatok kiterjedései is jól lettek transzformálva, illetve ezután készítettem én is neki egy PR-t az editor ikonokkal, illetve dinamikus láthatósággal és instancinggel. Ezt ő mergeelte is, és megtagelt az acknowledgementsben is.

Én pedig az ő implementációja alapján kipróbáltam a CompositorEffect-es megoldást, és tényleg sok problémát egyből megoldott. Így az uniózott projekt már azt használja a továbbiakban.

5.13.1. Ikonok, dinamikus láthatóság és instancing Pull Request

5.13.2. GLTF import Pull Request

5.13.3. Issue a gömbi harmonikusok helyes transzformációjáról

5.14. Kitérő: Typst

A diplomaterv dokumentumának elkészítéséhez a Typst nevű nyelvet használom, ami egy modern LaTeX megfelelő. Dani nevű szobatársam segítségével konvertáltuk a sablonokat (főleg ő), ez elérhető itt: [18].

5.14.1. Rövid bemutatás

open source, cég háttere, szintaktika, elterjedés

5.14.2. Miért jobb

word/libreoffice-nál, latexnél

Word: kód blokkok, számozás, képek számozása (?), bibliography beszúró GUI izébizénél sokkal egyszerűbb, stb

LaTeX: 20 package a fordításhoz, hosszú build time, csúnya szintaktika, végtelen mennyiségű fájl amik elég nehezen átláthatók, fordításkor is teleszemeteli a dolgot

összességében az élmény milyen

6. Fejezet

Plugin alkalmazásai, eredmények

6.1. Használati utasítás

Hogyan lehet betenni egy már meglevő projektbe, mi az h godot addon, stb stb

6.2. Autó szimulátor, mint példafelhasználás

leírni hogy mi ez a projekt, esetleg külön alfejezetben bemutatni [19]

18. ábrán az eredeti program látható

18. ábra: Példa képernyőkép a VitaVehicle autószimulátorról

A splat-os és az autós Godot projektek összefésülése egészen könnyen ment, a fájlok összemásolásán túl a `project.godot` fájlt kellett kompatibilissá tenni mindkét projekt részére, többek között a definiált (billentyűzet) bemenetet, és a globális scripteknek megfelelő scripteket (autoload) kellett átemelni. Szerencsére a `project.godot` egy szöveges fájlformátum, sima config fájlként viselkedik, így akár kézzel is könnyű volt az egyes bejegyzéseket átemelni, a maradék beállítást pedig könnyen megtaláltam az editorban.

Továbbá mindkét projektben volt az alapértelmezett jelenet, aminek a node-jaihoz a szükséges scriptek hozzá voltak adva. Kis logikázás után úgy láttam, hogy a splat-os projekt gyökérnode-ja kell, hogy maradjon, többek között azért is, hogy a hozzáadott SplatMesh-eket megtalálja a jelenetben. Nem volt különösebb gond az autós projekttel, ha annak a gyökérnode-ját a splat-os projekt gyökérnode-ja gyerekeként tettem be.

19. ábra: A VitaVehicle projekt összekombinálása a GS megjelenítővel

A 19. ábrán együtt látható a pontfelhős és a háromszöghálós autó, tesztelve a korrekt pozíciót és reakciót a környezetre

20. ábra: A kombinálás végső eredménye

20. ábrán látható a kombinálás végső eredménye: két, külön irányítható GS modell alapú autó (külön forgó, kormányozható kerekekkel), illetve egy plüssmaci, ami fizikailag reagál az autókra: eltolható, ha nekimegyünk.

6.2.1. Stuttering bug

Ami következményként nehézséget okozott, az a stuttering kiküszöbölése, amit az okozott, hogy az autós projektben a kamerát egy script állította, míg a splatos projektben ugyanezen (aktív) kamera transzformációját egy script továbbította a GPU-nak, és ez a két script valamiért nem meghatározott sorrendben futott. A process priority állításával a probléma megoldódott.

képek inkább ez után a fejezet után? lehet h jobban fel lehetne osztani ezt a részt

6.3. When Hamsters Attack TD, mint példafelhasználás

leírni hogy mi ez a projekt, esetleg külön alfejezetben bemutatni

játék bemutatása, integrációja a pluginnal, mint példa felhasználás [20]

erről részletesen beszélni (pl object pooling, animáció alkalmazása, ilyenek)

képek teamsről pl? vagy új screenshotok csinálása? (vszeg ennek a gépnek a screenshots mappájában is megvan)

6.4. Asset library plugin

Felkerül végre?? igen

leírni hogy milyen kontribúcióim vannak benne

6.5. Még mit lehetne értékelésnek, eredménynek?

objektív szempontok szerint összehasonlítani a többféle implementációt (2 féle relightolás, kínai vs saját branch, stb.)

igen pl a relightolást jól össze lehet hasonlítani, mert az 3 teljesen különböző megoldás

7. Fejezet

Összefoglaló

Sokat tanultunk a dipterv alatt, megismerkedtünk ezzel, azzal, godot rendering pipeline, gaussian splatting, fájlformátumok, typst, stb

a plugin használható, viszonylag jó integrációval, fenn van (??) az asset store-ban, hozzátettem bizonyos fejlesztéseket, mások meg itt és itt érhetők el, … (külön asset store item a relightolható mesh-esnek??, ha nem merge-eli majd?)

Irodalomjegyzék

[1] „Generatív AI és inverz módszerek a képszintézisben”. [Online]. Elérhető: https://vaitkusm.github.io/genai-inverse-graphics-bme-vik-iit/

[2] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, és George Drettakis, „3D Gaussian Splatting for Real-Time Radiance Field Rendering”, 2023, 2308.04079. [Online]. Elérhető: https://arxiv.org/abs/2308.04079

[3] Stefano Scolari, „Mesh2Splat: Fast mesh to 3D Gaussian splat conversion”. [Online]. Elérhető: https://github.com/electronicarts/mesh2splat

[4] „ReconWorldLab implementációja”. [Online]. Elérhető: https://github.com/ReconWorldLab/godot-gaussian-splatting

[5] G. Wu *és mtsai.*, „4D Gaussian Splatting for Real-Time Dynamic Scene Rendering”, 2024, 2310.08528. [Online]. Elérhető: https://arxiv.org/abs/2310.08528

[6] „Khronos Announces glTF Gaussian Splatting Extension”. [Online]. Elérhető: https://www.khronos.org/news/press/gltf-gaussian-splatting-press-release

[7] „GodotGaussianSplatting by 2Retr0”. [Online]. Elérhető: https://github.com/2Retr0/GodotGaussianSplatting

[8] „GodotGaussianSplatting by haztro”. [Online]. Elérhető: https://github.com/haztro/godot-gaussian-splatting

[9] „Gaussian splatting, native to Godot”. [Online]. Elérhető: https://klausi3d.github.io/godotGS/latest/

[10] „Collaboration request”. [Online]. Elérhető: https://github.com/ReconWorldLab/godot-gaussian-splatting/issues/1

[11] „Gaussian Splat Formats: PLY vs SPZ vs SOG vs KSplat”. [Online]. Elérhető: https://swyvl.io/blog/gaussian-splat-formats-ply-spz-ksplat/

[12] „glTF™ 2.0 Specification”. [Online]. Elérhető: https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html

[13] Wieland Morgenstern, Florian Barthel, Anna Hilsmann, és Peter Eisert, *Compact 3D Scene Representation via Self-Organizing Gaussian Grids*. Springer Nature Switzerland, 2025. [Online]. Elérhető: https://fraunhoferhhi.github.io/Self-Organizing-Gaussians/

[14] „SplatTransform”. [Online]. Elérhető: https://github.com/playcanvas/splat-transform

[15] „Vulkan Radix Sort”. [Online]. Elérhető: https://github.com/jaesung-cs/vulkan_radix_sort

[16] „Godot Gaussian Splatting Game”. [Online]. Elérhető: https://github.com/4321ba/GodotGaussianSplattingGame/tree/vitavehicle

[17] „GodotGaussianSplatting issue az AMD bughoz”. [Online]. Elérhető: https://github.com/2Retr0/GodotGaussianSplatting/issues/3

[18] „Typst BME Dipterv Sablon”. [Online]. Elérhető: https://codeberg.org/pilpol/bme-dipterv-sablon

[19] „VitaVehicle - "Realistic" Car Physics (g-rcp2/RCP4)”. [Online]. Elérhető: https://github.com/jreo03/g-rcp2/tree/godot-4.0.3-conversion

[20] „When Hamsters Attack TD”. [Online]. Elérhető: https://github.com/4321ba/WhenHamstersAttackTD

Függelék

A. További teendők a Dipterv 2-ig (végleges doksi leadásig)

- a WhenHamstersAttackTD [20] játék átírása félig (vagy teljesen) GSplatos modellek használatára
- ezen dokumentum finomítása, véglegesítése
  - irodalomjegyzék vs lábjegyzet? Melyiket mennyire? Mind mehet irodalomjegyzékbe, nem baj ha nagyon sok?
- TODO-k kijavítása, eltüntetése itt is és a kódban is
- minden kép, táblázat és kódrészlet be van számozva? és rendesen referálva?
- commit history-t, ai chat history-t, teams chatet visszaolvasni, hogy ne hagyjak ki semmit, amiről lehet írni

!!!kérdések:

- kódolás rész kész, más nem kell
- fejezetek sorrendje!
- irodalomjegyzék vs lábjegyzet
- kedvezményes tanrend nem kell
- terv szerint heti kb 5 oldal finomítása, mostmár inkább 10 xd (október eleje)

A.1. Egyéb TODO-k

TODO margó tükrözés rendesen, ha 2 oldalasat szeretnék, és a feladatkiíráshoz is a megfelelő margó! Illetve a legvégén: jól legyenek párosítva az oldalak, ha tényleg 2 oldalasat csinálok (tartalomjegyzék-tartalomjegyzék, absztrakt-abstract, stb.)

TODO Valahova a féléves beosztásokat: mi volt önlab1, mi dipterv1, mi dipterv2

TODO Daninak a sablonban levő módosításokat elküldeni, ha már kb fix

TODO jól elkülöníthető amit ténylegesen én csináltam, meg amit átvettem/amiről csak írok? ne időrendi sorrendben menjek a cuccokon végig, hanem logikai sorrendben.

TODO minden bibliography item fel lett használva valahol? footnote vs bibliography jól van kb elválasztva? (nincs többször ugyanaz a footnote, de a bibliography-k tényleg fontos h bibliography-k legyenek?)

TODO GH readme-be, illetve doksiba is (?) beírni, hogy melyik branch-en mi van. cleanup a githubon is, hogy értelmesen, jól nézzen ki.

Az egészet beadni AI-nak pl nyelvtani javításra (egy random helyen eglépelni, hogy észreveszi-e!!!)

időben elküldeni az egészet reviewra (pl november eleje-közepe)

tervezett ütemterv:

- végigmenni az egész szövegen ami van, megfelelő sorrendbe tenni a dolgokat, kb legyen meg a szöveg, struktúra
- AI táblázat betétele
- újra végigfutni mindenen, absztraktok, dolgozat szerkezete, egyéb TODO-k letisztázása
- képaláírások, kódaláírások rendben vannak? sehol sincs copy-paste ami nem oda tartozik?
- AI nyelvtani korrekció tesztje (viabilis megoldás-e, minél több hiba, magyartalan szöveg javítása)
- fentebbi todo-k átfutása, hogy van-e valami releváns
- elküldeni a konzulensnek, még mivel érdemes kiegészíteni, átrendezés, ilyesmi stb.
- legyen meg a szövegmennyiség kg-ra
- újabb nyelvtani ellenőrzés
- a/az az ábráknál / fejezetcímeknél kifejezetten! ezt már miután semmi átrendezés / új beszúrás nincs
- valamit tartalmilag totál elírni, hogy azt megtalálja-e egy AI, hha valahol totál marhaság lenne

beadás után nyomtatás, + a diasor elkészítése míg amíg jól emlékszem a dokumentumra!

ötletek a méret növelésére:

- nerf megemlítése / kijjebb fejtése
- irodalomkutatáshoz screenshotok, kipróbálni a dolgokat amiket referálok, illetve még írni róluk

TODO AI táblázat majd ide!